

AromaSDK



Lightweight development toolkit for cross-platform applications on Linux, Android, Windows, and embedded systems.

Version 0.0.1 (alpha)

Generated September 26, 2026

© 2026

Contents

Getting Started	3
Architecture Overview	4
Scene Graph & Nodes	5
Event System	6
Layout Engine	7
Rendering Pipeline	8
Animation Engine	9
Theming & Styling	10
Display & Feedback	11
Input & Controls	12
Layout & Navigation	13
Map Widget	14
Incense Declarative UI	15
Project Creation	16
Building & Deployment	17
SDK Downloads	18
Graphics Backends	19
Platform Backends	20
Font System	21
Testing & Quality	22
Glossary	23
API Reference	24

Getting Started

Start here. On Linux, the fastest path is the GUI installer. No cloning, no toolchain setup.

Recommended: download the GUI installer for Linux

[Download aroma-lavender-linux.tar.xz \(v0.0.2\)](#)

Or get it step by step:

1. Open the [releases page](#).
2. Download the Linux installer archive (`aroma-lavender-linux.tar.xz`).
3. Extract it: `bash tar -xf aroma-lavender-linux.tar.xz`
4. Run the installer, pick the components you want, and follow the screens.

What you need for the installer to run:

- GLIBC 2.27 or newer (check with `ldd --version`)
- OpenGL 2.1 or newer, or OpenGL ES 2.0 with EGL
- X11 (on Wayland it runs through XWayland)

If the installer does not start, check those three items first, then grab the newest release from the releases page.

Option A: Try Incense in your browser (no install)

1. Open `docs/sandbox.html` in this repo. You can also open the live sandbox from the docs home page.
2. Edit the code on the left.
3. Click Run.
4. The phone preview on the right updates at once.

Start with this sample:

```
Window {  
    width: 320  
    height: 480  
    title: "My App"  
  
    Container {  
        x: 0  
        y: 0  
        width: 320  
        height: 480  
        layout: flex  
        direction: column  
  
        Label {  
            text: "Hello"  
            style: large  
        }  
  
        Button {  
            text: "Click Me"  
            x: 20  
            y: 100  
            width: 120  
            height: 40  
            on_click: "handle_click"  
        }  
    }  
}
```

If the preview stays blank, click Run again and check the status bar at the bottom. It shows the exact line with the error.

Option B: Build from source on Linux

Use this path if you want the latest code or you plan to change the framework itself. Otherwise the installer above is quicker.

What you need

- Linux with Git, CMake 3.22.1 or newer, and Python 3.8 or newer
- A C compiler such as GCC or Clang
- Optional for Android builds: JDK 17 plus Android SDK, NDK, and CMake

Check your tools first:

```
cmake --version
python3 --version
gcc --version
```

1. Clone the repo

```
git clone https://github.com/BinaryInkTN/AromaUI.git --recursive
cd AromaUI
```

You need `--recursive`. It pulls in FreeType and other vendored deps.

2. Check your setup

```
python3 bin/aroma doctor
```

Read the output. Fix any item marked FAIL before you go on. For Android, install the SDK and NDK with Android Studio or the command line tools, then run `doctor` again.

3. Create a project

```
python3 bin/aroma create MyApp
cd MyApp
```

You get this layout:

- `src/main.c` : app entry point
- `CMakeLists.txt` : build config for Linux and Web
- `android/` : Android project with Gradle and JNI bridge

4. Build and run on Linux

Run this inside your new `MyApp` folder:

```
python3 ../bin/aroma build linux
python3 ../bin/aroma run linux
```

Or build by hand:

```
mkdir build && cd build
cmake ..
make -j$(nproc)
./MyApp
```

A window opens with "Hello, World!" and a counter that updates each second. If the build fails, see Common errors below.

5. Build for Web

You need the Emscripten SDK under `vendors/emscripten` or `~/.aroma/vendors/emscripten`.

```
python3 ../bin/aroma build web
python3 -m http.server 8080 --directory build_web
```

Open `http://localhost:8080` in your browser.

6. Build for Android

You need a working SDK, NDK, and an emulator or a plugged in device.

```
python3 ../bin/aroma build android  
python3 ../bin/aroma run android --emu
```

Use `--release` or `--aab` only for store builds.

Your first C app

`aroma create` gives you this working program in `src/main.c`:

```

#include <aroma.h>
#include <unistd.h>

int main(void)
{
    aroma_ui_init();

    AromaTheme theme = aroma_theme_create_material_blue_dark();
    aroma_ui_set_theme(&theme);

    AromaWindow *window = aroma_ui_create_window("Hello World", 400, 600);

    AromaFont *font = aroma_font_create_from_memory(
        aroma_ubuntu_ttf,
        aroma_ubuntu_ttf_len,
        24);

    AromaNode *root = aroma_ui_container(
        (AromaNode *)window,
        0, 0, 400, 600,
        AROMA_LAYOUT_MODE_FLEX,
        AROMA_FLEX_COLUMN,
        AROMA_JUSTIFY_CENTER,
        AROMA_ALIGN_CENTER);

    aroma_ui_label(
        root,
        "Hello, World!",
        0, 0,
        LABEL_STYLE_LABEL_LARGE,
        font);

    while (aroma_ui_is_running())
    {
        aroma_ui_process_events();
        aroma_ui_render(window);
        usleep(1000000);
    }

    aroma_font_destroy(font);
    aroma_ui_destroy_window(window);
    aroma_ui_shutdown();
}

```

```
    return 0;
}
```

The pattern is always the same:

1. Call `aroma_ui_init()`.
2. Create a window.
3. Add nodes such as containers and labels.
4. Loop with `aroma_ui_process_events()` and `aroma_ui_render()`.
5. Clean up with `aroma_ui_shutdown()`.

Common errors

- `cmake: command not found`: install CMake, then open a new terminal.
- Empty clone or missing headers: you cloned without `--recursive`. Run `git submodule update --init --recursive`.
- `Emscripten SDK not found`: place the SDK under `vendors/emscripten` or `~/.aroma/vendors/emscripten`, then retry `build web`.
- `No Android SDK platforms found`: set `ANDROID_HOME` or install platforms with the SDK Manager, then retry.
- `Not an Aroma project (no android/ directory)`: run Android commands inside the folder created by `aroma create`.
- Sandbox shows an error: read the status bar. Fix the line it points to, then click Run.

What's next

- Read [Architecture Overview](#) to learn how nodes, layout, and rendering fit together.
- Browse [Input and Controls](#) for buttons, text, sliders, and lists.
- Open [Incense Declarative UI](#) to prototype screens fast, then port them to C.

- See [Building and Deployment](#) for Linux, Android, and Web builds in full detail.

Architecture Overview

AromaUI has four layers. Learn these layers once and the rest of the framework makes sense.

System Layers

📄 flowchart TD

flowchart TD

```
App["Application Layeraroma_ui_button(), aroma_ui_label()"]
Core["Core FrameworkAromaNode, Layout, Events, DrawList"]
ABI["Backend Abstraction LayerAromaBackendABI proxy"]
Backends["Platform & Graphics BackendsGLS3, Vulkan, TFT_eSPI, Android"]
```

App --> Core

Core --> ABI

ABI --> Backends

1. **Application Layer:** your code. Call factory functions from `include/aroma_ui.h` to build the scene graph.
2. **Core Framework:** shared logic. It owns the `AromaNode` tree, layout, events, and rendering.
3. **ABI:** a switchboard. It sends generic draw calls to the active backend.
4. **Backends:** platform code. It handles windows, input, and GPU output.

Key Concepts

AromaNode

Each visible item is an `AromaNode`. Nodes form a tree. Each parent holds up to 64 children. Each node stores:

- Geometry (`AromaRect`)
- Layout hints (`AromaLayout`)
- Visibility and Z-index
- A `draw_cb` function for rendering

Dirty Region Tracking

Only changed nodes redraw. After you change a property, call `aroma_node_invalidate()`. The framework tracks dirty nodes and skips the rest.

DrawList

Widgets do not draw at once. They record commands into an `AromaDrawList`. On flush, the list sorts by Z-index and sends batches to the GPU. This gives you:

- Correct Z order in any tree order
- Skipped offscreen nodes
- Fast batches on small hardware

Data Flow: A Button Click

📄 flowchart LR

flowchart LR

```

    Touch["User Touch"] --> Platform["Platform Backendcaptures event"]
    Platform --> EventSys["Event Systemqueues + hit-tests"]
    EventSys --> Button["Button listener fires"]
    Button --> Callback["Your callback runs"]
    Callback --> Invalidate["aroma_node_invalidate(button)"]
    Invalidate --> Frame["Next frame:layout + draw"]
    Frame --> DrawList["DrawList records commands"]
    DrawList --> ABI["ABI routes toGLES3/Vulkan/TFT"]
    ABI --> Screen["Screen updates"]
  
```

File Map

Area	Key Files
Entry points	<code>include/aroma_ui.h</code> , <code>src/core/aroma_ui_impl.c</code>
Node system	<code>include/aroma_node.h</code> , <code>src/core/aroma_node.c</code>
Layout	<code>src/core/aroma_layout.c</code>
Events	<code>src/core/aroma_event.c</code>
Rendering	<code>src/core/aroma_drawlist.c</code> , <code>src/backends/aroma_abi.c</code>
Graphics	<code>src/backends/graphics/aroma_graphics_gles3.c</code>
Platforms	<code>src/backends/platforms/aroma_platform_glfw.c</code>

What's Next

- Read [Scene Graph](#) for node lifecycle.
- Read [Events](#) for input flow.
- Read [Rendering Pipeline](#) for draw details.

Scene Graph & Nodes

Every UI element in AromaUI is an `AromaNode`. Nodes form a tree, and the framework traverses this tree for layout, hit-testing, and rendering.

The AromaNode Struct

```
typedef struct AromaNode {
    uint64_t node_id;
    AromaNodeType node_type;           // ROOT, CONTAINER, WIDGET
    int32_t z_index;
    AromaRect rect;                    // x, y, width, height
    AromaLayout layout;                // layout hints
    bool visible;
    bool is_dirty;
    bool subtree_dirty;
    AromaNode *parent_node;
    AromaNode *child_nodes[64];       // max 64 children
    int child_count;
    void *node_widget_ptr;            // widget-specific data
    AromaNodeDrawFn draw_cb;
    // ... padding for alignment
} AromaNode;
```

Key limits: - **64 children per node** - fixed-size array avoids dynamic allocation - **64 properties per widget** - enforced by the Incense loader - **1024 dirty nodes per frame** - global dirty list capacity

Node Lifecycle

Creation

```
AromaNode *node = aroma_node_create(parent, NODE_TYPE_WIDGET, x, y, w, h);
```

Nodes are allocated from a slab allocator. For embedded targets (ESP32), this avoids heap fragmentation. For desktop/web, standard `malloc` is used as a fallback.

Parenting

```
aroma_node_add_child(parent, child);
```

- Child limit is enforced at 64.
- `subtree_dirty` propagates up to the root on invalidation.

Destruction

```
aroma_node_destroy(node);
```

Recursively destroys all children. Widgets can provide a `destroy_cb` to free internal state.

Dirty-Region Tracking

Only dirty nodes are processed during layout and rendering.

```
void aroma_node_invalidate(AromaNode *node);
```

Sets `is_dirty = true` on the node and `subtree_dirty = true` on all ancestors. The renderer skips branches where `subtree_dirty` is false.

Memory Management

Slab Allocator

For embedded targets, AromaUI uses a slab allocator with pools ranging from 32 bytes to 2048 bytes. Widget data (e.g., `AromaListViewInternal`) is allocated from these pools.

⬡ flowchart LR

flowchart LR

```
Request["aroma_widget_alloc(size)"] --> Check{"size <= 2048?"}
Check -->|Yes| Bucket["Find bucket index"]
Bucket --> Slab["__slab_pool_alloc()"]
Slab --> Pool["AromaSlabAllocator"]
Check -->|No| Malloc["standard malloc()"]
```

Alignment

`AromaNode` includes explicit padding to ensure 8-byte alignment, preventing bus errors on RISC architectures and WASM.

Internal APIs

The framework exposes these lower-level functions for advanced use cases and internal machinery:

Function	Role
<code>aroma_node_create()</code>	Allocates a node from the slab allocator
<code>aroma_node_add_child()</code>	Links a child into the parent's fixed-size array
<code>aroma_node_remove_child()</code>	Shifts siblings to maintain array density
<code>aroma_node_destroy()</code>	Recursively frees a subtree
<code>aroma_node_invalidate()</code>	Marks a node dirty and propagates <code>subtree_dirty</code> upward
<code>aroma_node_set_layout_none()</code>	Absolute positioning
<code>aroma_node_set_layout_fill()</code>	Match parent bounds
<code>aroma_node_set_layout_center()</code>	Center within parent
<code>aroma_node_set_z_index()</code>	Controls draw order
<code>aroma_node_set_hidden()</code>	Toggles visibility without destruction

Most applications should use the higher-level factory functions in `include/aroma_ui.h` instead of calling these directly.

What's Next

- Learn how [Events](#) are dispatched to nodes.
- Understand [Layout](#) calculation.
- See how nodes become pixels in the [Rendering Pipeline](#).

Event System

AromaUI's event system captures raw platform input and dispatches it to the correct `AromaNode`. It supports hit-testing, priority-based listeners, event bubbling, and touch-to-scroll interception.

Event Lifecycle

📄 flowchart LR

flowchart LR

```
Platform["Platform captures raw input"] --> Create["aroma_event_create() allocates event"]
Create --> Queue["aroma_event_queue() adds to thread-safe queue"]
Queue --> Process["aroma_event_process_queue() drains queue"]
Process --> Dispatch["aroma_event_dispatch() finds target"]
Dispatch --> Listeners["Listeners execute (sorted by priority)"]
Listeners --> Consumed{"Consumed?"}
Consumed -->|No| Bubble["Bubble to parent"]
Consumed -->|Yes| End["End propagation"]
```

Hit-Testing

When a pointer event arrives, `aroma_event_hit_test()` recursively searches the scene graph from the root. It respects: - **Z-order** - top-most nodes are tested first - **Visibility** - hidden nodes are skipped - **Clipping** - nodes outside their parent's clip rect are ignored

Event Listeners

Register callbacks on any node:

```
aroma_event_subscribe(node_id, EVENT_TYPE_MOUSE_CLICK, handler, user_data);
```

Parameter	Description
<code>node_id</code>	Target node (returned by <code>aroma_node_create</code>)
<code>event_type</code>	<code>EVENT_TYPE_MOUSE_CLICK</code> , <code>EVENT_TYPE_TOUCH_DOWN</code> , etc.
<code>handler</code>	<code>bool (*)(AromaNode *target, AromaEvent *event, void *user_data)</code>
<code>user_data</code>	Optional context passed to the handler

Return `true` from the handler to consume the event and stop bubbling.

Scroll Interception

Scrollable containers (like `AromaContainer`) intercept touch events to implement kinetic scrolling:

1. `TOUCH_DOWN` - identifies scrollable ancestor
2. `TOUCH_MOVE` - tracks delta; if movement exceeds 8px threshold, intercepts
3. Once intercepted, child nodes receive `TOUCH_CANCEL` and the container handles scrolling

Memory: Static Event Pool

Events are allocated from a fixed pool of 256 entries (`g_event_pool`). This guarantees O(1) allocation and zero heap fragmentation - critical for ESP32 and WASM targets.

Internal APIs

The event system exposes these lower-level functions for framework internals and advanced use cases:

Function	Role
<code>aroma_event_subscribe()</code>	Registers a listener on a node for a specific event type
<code>aroma_event_unsubscribe()</code>	Removes a previously registered listener
<code>aroma_event_create()</code>	Allocates an event from the static pool
<code>aroma_event_queue()</code>	Adds an event to the thread-safe processing queue
<code>aroma_event_process_queue()</code>	Drains the queue and dispatches events
<code>aroma_event_dispatch()</code>	Hit-tests and delivers events to target nodes
<code>aroma_event_hit_test()</code>	Recursive scene-graph search for pointer targets

Application code typically interacts with events through widget-specific callbacks (e.g., `aroma_button_set_on_click()`) rather than calling these functions directly.

What's Next

- Learn how [Layout](#) positions nodes.
- See how events interact with [Rendering](#).
- Explore [Animation](#) for property transitions.

Layout Engine

The layout engine calculates the screen-space position and size of every `AromaNode`. It runs every frame for dirty subtrees and supports absolute positioning, flexbox, and grid layouts.

Layout Types

Each node has an `AromaLayout` struct that determines how it positions itself relative to its parent.

Self-Layout (how a node positions itself)

Mode	API	Description
None	<code>aroma_node_set_layout_none(node, x, y, w, h)</code>	Absolute positioning
Fill Parent	<code>aroma_node_set_layout_fill(node)</code>	Match parent bounds exactly
Center	<code>aroma_node_set_layout_center(node, w, h)</code>	Center within parent
Anchor	<code>aroma_node_set_layout_anchor(node, left, top, right, bottom)</code>	Pin to edges; supports stretching

Container Layout (how a node arranges its children)

Set via `aroma_node_set_layout_mode(node, mode)`:

Mode	Description
<code>AROMA_LAYOUT_MODE_NONE</code>	Children use their own self-layout
<code>AROMA_LAYOUT_MODE_FLEX</code>	Flexbox: row or column layout
<code>AROMA_LAYOUT_MODE_GRID</code>	Equal-sized cells in row-major order

Flexbox Layout

The flex implementation supports:

- **Direction:** `AROMA_FLEX_ROW` or `AROMA_FLEX_COLUMN`
- **Justify:** `start`, `center`, `end`, `space_between`, `space_around`, `space_evenly`
- **Align:** `start`, `center`, `end`, `stretch`
- **Flex Grow:** Children with `flex_grow > 0` absorb remaining space

```

aroma_node_set_layout_mode(container, AROMA_LAYOUT_MODE_FLEX);
aroma_node_set_flex_direction(container, AROMA_FLEX_COLUMN);
aroma_node_set_justify_content(container, AROMA_JUSTIFY_START);
aroma_node_set_align_items(container, AROMA_ALIGN_STRETCH);

```

Grid Layout

```

aroma_node_set_layout_mode(container, AROMA_LAYOUT_MODE_GRID);
aroma_container_set_grid_cols(container, 3);
aroma_container_set_grid_rows(container, 2);

```

Children are placed into equal-sized cells automatically.

Scrollable Containers

`AromaContainer` can act as a scrollable viewport:

```
AromaNode *sc = aroma_container_create(parent, x, y, w, h);
aroma_container_set_scrollable(sc, true);
aroma_container_set_scroll_direction(sc, AROMA_SCROLL_VERTICAL);
```

The container tracks `scroll_fx` / `scroll_fy` offsets, which are subtracted from child coordinates during layout and rendering. It includes:

- **Velocity tracking** for fling gestures
- **Overscroll bounce** with spring-back animation
- **Auto content sizing** based on children

Layout Update Flow

📄 flowchart TD

flowchart TD

```
Start["aroma_layout_update()"] --> Dirty{"Node dirty?"}
Dirty -->|No| Skip["Skip subtree"]
Dirty -->|Yes| Self["Resolve self-layout(calculate node rect)"]
Self --> Container{"Has children?"}
Container -->|No| Done["Write AromaRect"]
Container -->|Yes| Mode{"Container mode?"}
Mode -->|NONE| Absolute["Children use own self-layout"]
Mode -->|FLEX| Flex["Flexbox: main/cross axis"]
Mode -->|GRID| Grid["Grid: row-major cells"]
Absolute --> Done
Flex --> Done
Grid --> Done
Done --> Next["Next dirty node"]
```

1. `aroma_layout_update()` is called during the frame update
2. For each dirty node, resolve self-layout first (calculate node's own rect)
3. Then resolve container layout (position children)

4. Write final pixel values into `AromaRect`

Key APIs

These functions control node layout behavior:

Function	Purpose
<code>aroma_node_set_layout_none(node, x, y, w, h)</code>	Absolute positioning
<code>aroma_node_set_layout_fill(node)</code>	Match parent bounds exactly
<code>aroma_node_set_layout_center(node, w, h)</code>	Center within parent
<code>aroma_node_set_layout_anchor(node, l, t, r, b)</code>	Pin to edges with optional stretch
<code>aroma_node_set_layout_mode(node, mode)</code>	Set container layout mode (none/flex/grid)
<code>aroma_node_set_flex_direction(node, dir)</code>	Set flex direction (row/column)
<code>aroma_node_set_justify_content(node, mode)</code>	Set main-axis alignment
<code>aroma_node_set_align_items(node, mode)</code>	Set cross-axis alignment
<code>aroma_container_set_scrollable(node, bool)</code>	Enable scroll viewport behavior
<code>aroma_container_set_scroll_direction(node, dir)</code>	Lock scroll axis
<code>aroma_container_update_auto_content_size(node)</code>	Recalculate scroll bounds from children

Higher-level factory functions in `include/aroma_ui.h` wrap many of these calls.

What's Next

- Learn how nodes are rendered in the [Rendering Pipeline](#).
- See how [Events](#) interact with scroll containers.
- Explore [Theming](#) for visual customization.

Rendering Pipeline

AromaUI uses deferred rendering. Instead of drawing immediately, widgets record commands into an `AromaDrawList`. At the end of each frame, commands are sorted by Z-index and flushed to the graphics backend.

Frame Lifecycle

📄 flowchart TD

flowchart TD

```

A["1. Invalidationnodes marked dirty"] --> B["2. Layoutrecalculate dirty sub"]
B --> C["3. Collectiondraw_cb records tasks"]
C --> D["4. Sortingqsort by z_index"]
D --> E["5. Recordingtasks execute to DrawList"]
E --> F["6. Flushdispatch to ABI"]
F --> G["7. Swappresent framebuffer"]

```

DrawList Benefits

- **Z-sorting:** Nodes with higher `z_index` draw on top, regardless of tree order
- **Batching:** Multiple rectangles can be merged into a single `glDrawArrays` call
- **Frustum culling:** Offscreen nodes are skipped during collection

AromaBackendABI

The ABI sits between the core framework and hardware backends. When a DrawList is active, it records commands. Otherwise, it passes calls directly to the backend (immediate mode).

```
// Typical widget draw callback
void aroma_button_draw(AromaNode *node, size_t window_id) {
    AromaGraphicsInterface *gfx = aroma_graphics_get_interface(window_id);
    gfx->fill_rectangle(window_id, rect.x, rect.y, rect.w, rect.h, bg_color, true, radius);
    gfx->render_text(window_id, font, text, tx, ty, text_color);
}
```

Backend Flush

During `aroma_drawlist_flush()`: 1. Iterate buffered commands 2. Dispatch to the active `AromaGraphicsInterface` (GL ES3, Vulkan, or TFT_eSPI) 3. For TFT_eSPI: use smart tiling - only flush dirty horizontal tiles to avoid full-screen buffer allocation

Batching Optimizations

GL ES3 Shape Batching

The GL ES3 backend buffers up to 256 quads before issuing a draw call. This reduces GPU state changes for UIs with many simple rectangles.

Font LRU Cache

Glyph textures are cached per-window (max 16 fonts). When the cache is full, the least recently used font is evicted.

Implementation Notes

The rendering pipeline relies on these internal mechanisms:

Concern	How It Works
Force redraw	<code>aroma_ui_request_redraw(window)</code> marks the window dirty
Draw order	Higher <code>z_index</code> draws later (on top)
Clip region	Set via <code>gfx->set_clip(rect)</code> before drawing
Backend selection	Configured at platform init (<code>aroma_backend_abi_init</code>)

Most render control happens automatically through the DrawList and dirty-region system. Direct manipulation is rarely needed in application code.

What's Next

- Learn about [Animation](#) for smooth property transitions.
- Explore [Theming](#) for visual customization.
- See [Scene Graph](#) for node lifecycle details.

Animation Engine

AromaUI includes a lightweight animation engine for smooth property transitions. It interpolates values over time using easing functions and integrates with the dirty-region system to trigger redraws automatically.

Quick Start

```
// Animate a node sliding in from the left
aroma_animation_start(node, AROMA_ANIM_SLIDE_X, -100.0f, 0.0f, 300);

// Fade out over 500ms
aroma_animation_start(node, AROMA_ANIM_FADE, 1.0f, 0.0f, 500);

// Custom animation (e.g., color blend)
static uint32_t s_anim_from;
static uint32_t s_anim_to;

static void on_tint_step(AromaNode *n, float current_val, void *ud) {
    (void)ud;
    aroma_node_set_bg_color(n, aroma_color_blend(s_anim_from, s_anim_to, current_val));
}

aroma_animation_start_custom(node, 0.0f, 1.0f, 400, on_tint_step, NULL);
```

Animation Types

Type	Property Modified
AROMA_ANIM_SLIDE_X	node->rect.x
AROMA_ANIM_SLIDE_Y	node->rect.y
AROMA_ANIM_SCALE_X	node->rect.width
AROMA_ANIM_SCALE_Y	node->rect.height
AROMA_ANIM_FADE	node->opacity
AROMA_ANIM_CUSTOM	User-defined callback

Easing Functions

Easing	Effect
AROMA_EASE_LINEAR	Constant speed
AROMA_EASE_IN_QUAD	Slow start
AROMA_EASE_OUT_QUAD	Fast start, slow end
AROMA_EASE_OUT_CUBIC	Smooth deceleration (default)
AROMA_EASE_OUT_BACK	Slight overshoot before settling
AROMA_EASE_OUT_ELASTIC	Damped oscillation

How It Works

📄 flowchart LR

flowchart LR

```

    Start["aroma_animation_start()"] --> List["Add to global linked list"]
    List --> Timer["16ms timer triggers"]
    Timer --> Progress["Calculate progress(now - start) / duration"]
    Progress --> Ease["Apply easing function"]
    Ease --> Write["Write interpolated value to node property"]
    Write --> Invalidate["aroma_node_invalidate() mark node dirty"]
    Invalidate --> Done{"Complete?"}
    Done -->|No| Timer
    Done -->|Yes| Remove["Remove from list"]
  
```

Stopping Animations

```
aroma_animation_stop(node); // stops all animations on this node
```

If a node already has a running animation, `aroma_animation_start()` replaces it.

Integration with Layout

Animations update node properties (position, size, opacity) directly. The layout engine picks up these changes during the next frame's dirty-subtree traversal. No manual layout calls are needed.

Animation APIs

The animation engine exposes these functions for property transitions:

Function	Purpose
<code>aroma_animation_start(target, type, start, end, duration_ms)</code>	Begin a built-in animation (slide, scale, fade)
<code>aroma_animation_start_custom(target, duration_ms, callback, ud)</code>	Begin a user-defined animation
<code>aroma_animation_stop(target)</code>	Cancel all active animations on a node

Animations update node properties directly and integrate with the dirty-region system. The layout engine picks up changes automatically during the next frame.

What's Next

- Learn how [Layout](#) resolves node positions.
- See how [Rendering](#) draws animated nodes.
- Explore [Theming](#) for visual design.

Theming & Styling

AromaUI's theming system provides centralized control over colors, typography, spacing, and shadows. It supports built-in presets (Material, Dark, High Contrast) and runtime theme switching.

Quick Start

```
// Use a built-in dark theme
AromaTheme theme = aroma_theme_create_dark();
aroma_ui_set_theme(&theme);

// Customize colors
theme.colors.background = 0xFF1A1A1A;
theme.colors.primary = 0xBB86FC;

// Adjust spacing
theme.spacing.padding = 16;
theme.spacing.border_radius = 8.0f;

// Apply to a specific widget style
AromaStyle style = aroma_style_create_from_theme(&theme);
aroma_button_set_style(btn, &style);
```

Theme Structure

Struct	Purpose	Key Fields
<code>AromaColorPalette</code>	Color scheme	<code>primary</code> , <code>background</code> , <code>surface</code> , <code>text_primary</code> , <code>border</code>
<code>AromaSpacing</code>	Layout metrics	<code>padding</code> , <code>margin</code> , <code>border_radius</code> , <code>border_width</code>
<code>AromaTypography</code>	Text appearance	<code>font_size</code> , <code>line_height</code> , <code>font_name</code> , <code>font_color</code>
<code>AromaTheme</code>	Top-level container	Combines Palette, Spacing, Typography
<code>AromaStyle</code>	Component styling	State colors (<code>idle</code> , <code>hover</code> , <code>active</code>), shadow

Built-in Presets

Function	Description
<code>aroma_theme_create_default()</code>	Standard light theme
<code>aroma_theme_create_dark()</code>	Standard dark theme
<code>aroma_theme_create_material_preset(preset)</code>	Material light theme (6 color variants)
<code>aroma_theme_create_material_preset_dark(preset)</code>	Material dark theme

Material presets: `PURPLE`, `BLUE`, `TEAL`, `GREEN`, `ORANGE`, `PINK`

Color Utilities

```
// Adjust brightness (factor > 0 lightens, < 0 darkens)
uint32_t lighter = aroma_color_adjust(color, 0.2f);

// Blend two colors
uint32_t blended = aroma_color_blend(color1, color2, 0.5f);

// Pack RGBA
uint32_t rgba = aroma_color_rgba(0xFF, 0x00, 0x00, 0xFF);
```

Shadows

```
AromaShadow shadow = aroma_shadow_create_soft();
aroma_style_apply_shadow(&style, &shadow);
```

Presets: `aroma_shadow_create_soft()`, `aroma_shadow_create_subtle()`,
`aroma_shadow_create_deep()`

How Themes Apply

1. Set a global theme with `aroma_ui_set_theme(&theme)`
2. `aroma_ui_set_theme()` invalidates every window and requests a redraw, so the switch paints on the next frame with no manual tree walk
3. Widgets follow the live global theme on every draw; widgets that cache derived colors keep a `use_theme_colors` flag and refresh from the current theme unless you overrode them with an explicit setter (e.g. `aroma_button_set_colors()`, `aroma_listview_set_header_colors()`, `aroma_gauge_set_colors()`), which opts that widget out
4. Call `aroma_style_create_from_theme(&theme)` to map theme colors to a widget's state colors
5. The widget uses these colors during its `draw_cb`

Helpers for theme-aware code:

```
uint64_t v0 = aroma_theme_get_version();
aroma_ui_set_theme(&black);
bool changed = (aroma_theme_get_version() != v0); /* true */

if (aroma_theme_is_dark(NULL)) { /* NULL selects the global theme */
    /* pick inverse-surface colors */
}
```

Theming APIs

The theme system exposes these functions for runtime customization:

Function	Purpose
<code>aroma_ui_set_theme(&theme)</code>	Install a global theme; invalidates all windows and redraws
<code>aroma_theme_get_version()</code>	Generation counter, bumped on every theme switch
<code>aroma_theme_is_dark(theme)</code>	True when the theme background is dark (<code>NULL</code> = global)
<code>aroma_theme_create_custom()</code>	Zero-initialized theme to fill in from scratch
<code>aroma_style_create_from_theme(&theme)</code>	Map theme colors to a widget style (returns <code>AromaStyle</code>)
<code>aroma_color_blend(c1, c2, factor)</code>	Linearly interpolate between two colors
<code>aroma_color_adjust(color, factor)</code>	Brighten or darken a color
<code>aroma_shadow_create_soft/subtle/deep()</code>	Generate shadow presets
<code>aroma_style_apply_shadow(&style, &shadow)</code>	Bind shadow to a widget style

Widgets follow the live global theme on every draw. Override per-widget appearance with an explicit setter (e.g. `aroma_button_set_colors()`); the override persists across theme switches for that widget.

Live demo

Toggle the docs light/dark switch and watch this preview follow it:

Live demo [Open in sandbox](#)

I

What's Next

- Explore [Widget Library](#) to see theming in action.
- Learn [Rendering](#) for draw optimization.
- Check [Animation](#) for property transitions.

Display & Feedback

Visual and informational widgets: labels, images, cards, progress indicators, snackbars, and dialogs.

Labels and Icons

```
AromaNode *label = aroma_ui_label(root, "Hello World", 20, 20, LABEL_STYLE_LARGE, font);  
AromaNode *icon = aroma_ui_icon(root, AROMA_ICON_HOME, 100, 20, 32, 0xFF0000);
```

- LABEL_STYLE_LARGE, LABEL_STYLE_MEDIUM, LABEL_STYLE_SMALL
- Icons render a single glyph from the icon font

Images and GIFs

```
AromaNode *img = aroma_ui_image(root, "assets/photo.png", 0, 0, 200, 150);  
aroma_gif_play(gif_node); // start animation
```

Images are loaded via `stb_image` and cached as GPU textures. GIFs maintain a frame timer for automatic playback.

Cards

```
AromaNode *card = aroma_ui_card(root, 20, 20, 300, 100, CARD_TYPE_ELEVATED);
```

Type	Description
<code>CARD_TYPE_FILLED</code>	Solid background blended with primary color
<code>CARD_TYPE_TONAL</code>	Subtle variant of filled
<code>CARD_TYPE_GLASS</code>	Frosted glass: blurs the backdrop in place, then draws a translucent tint with glossy edge
<code>CARD_TYPE_OUTLINED</code>	Hollow with border

Frosted glass

Glass cards blur whatever was rendered behind them (backdrop blur) before drawing their translucent tint, so they read as frosted glass instead of flat translucency:

```
AromaNode *glass = aroma_ui_frosted_card(root, 20, 20, 300, 100, 14.0f);
aroma_card_set_backdrop_blur(glass, 20.0f); /* retune, 0 disables */
```

- Blur radius is in pixels. Glass cards default to a 14px blur; other card types default to 0 (no blur).
- Backdrop blur needs graphics-backend support (GLES3 implements it; query with `aroma_graphics_supports_backdrop_blur()`). Unsupported backends skip the blur and still draw the tint, so glass degrades gracefully.
- The blur runs in draw order between background and foreground, and honors the card's rounded corners.
- One backdrop snapshot is shared per frame: stacked glass surfaces all blur the same backdrop, so a sheet over a card (or pill over drawer) adds tint without re-blurring and darkening the glass below. The snapshot is captured on first use each frame, so blur is correct from the very first frame, and is sampled mipmapped for a wide, smooth falloff.

Progress Indicators

```
AromaNode *bar = aroma_ui_progressbar(root, 20, 200, 260, 4, 0.75f);
AromaNode *gauge = aroma_ui_gauge(root, 300, 20, 120, 120, 0.6f);
```

- ProgressBar: linear, 0.0 to 1.0 value
- Gauge: circular arc with needle

Snackbar

```
aroma_snackbar_show(window, "Item deleted", 3000, "UNDO", on_undo_callback);
```

Snackbars appear at the bottom with an optional action button. They auto-dismiss after the duration (ms).

Dialog

```
AromaNode *dialog = aroma_ui_dialog(root, "Confirm", "Delete this item?", 320, 180);
aroma_dialog_add_action(dialog, "Cancel", DIALOG_ACTION_CANCEL, on_cancel);
aroma_dialog_add_action(dialog, "Delete", DIALOG_ACTION_DESTRUCTIVE, on_delete);
```

Dialogs are modal overlays with up to 3 action buttons.

Live demo [Open in sandbox](#)

I

Live demo

Live demo [Open in sandbox](#)

I

Live demo [Open in sandbox](#)

I

What's Next

- Learn [Input & Controls](#) for interactive widgets.
- Explore [Layout & Navigation](#) for containers and scrolling.
- Try [Incense](#) for rapid UI prototyping.

Input & Controls

Interactive widgets: buttons, checkboxes, switches, sliders, textboxes, and chips.

Buttons

```
static bool on_button_click(AromaNode *node, void *ud) {
    (void)node;
    (void)ud;
    /* handle click */
    return true;
}

AromaNode *btn = aroma_ui_button(root, "Click Me", 20, 20, 160, 48,
                                on_button_click, NULL, NULL);
```

Variants: `standard`, `filled`, `tonal`, `outlined`

Icon Button

```
AromaNode *ib = aroma_ui_icon_button(root, AROMA_ICON_SETTINGS, 300, 20, 48, 48);
```

Checkbox

```
static void on_checkbox_toggle(bool checked, void *ud) {
    (void)checked;
    (void)ud;
    /* handle toggle */
}

AromaNode *cb = aroma_ui_checkbox(root, "Enable notifications", 20, 100, 300, 32,
                                   on_checkbox_toggle, NULL, NULL);
```

Radio Button

```
#define SETTINGS_GROUP 1

AromaNode *rb1 = aroma_ui_radiobutton(root, "Option A", 20, 140, 200, 32,
                                       SETTINGS_GROUP, NULL, NULL, NULL);
AromaNode *rb2 = aroma_ui_radiobutton(root, "Option B", 20, 180, 200, 32,
                                       SETTINGS_GROUP, NULL, NULL, NULL);
```

Radio buttons in the same group are mutually exclusive.

Switch

```
static bool on_switch_toggle(AromaNode *node, void *ud) {
    (void)node;
    (void)ud;
    /* handle toggle */
    return true;
}

AromaNode *sw = aroma_ui_switch(root, 20, 220, 56, 28, true,
                                on_switch_toggle, NULL);
```

Slider

```
static bool on_slider_change(AromaNode *node, void *ud) {
    (void)node;
    (void)ud;
    /* handle value change */
    return true;
}

AromaNode *slider = aroma_ui_slider(root, 20, 280, 260, 32, 0, 100, 50,
                                     on_slider_change, NULL);
```

Textbox

```
static bool on_text_change(AromaNode *node, const char *text, void *ud) {
    (void)node;
    (void)text;
    (void)ud;
    /* handle text change */
    return true;
}

AromaNode *tb = aroma_ui_textbox(root, 20, 340, 260, 48, "Enter name...",
                                  on_text_change, NULL, NULL);
```

Chip

```
AromaNode *chip = aroma_ui_chip(root, "Filter", 20, 400, 100, 36, CHIP_TYPE_FILTER);
aroma_chip_set_selected(chip, true);
```

Type	Use Case
<code>CHIP_TYPE_ACTION</code>	Single action
<code>CHIP_TYPE_CHOICE</code>	Selection from options
<code>CHIP_TYPE_FILTER</code>	Toggleable filter
<code>CHIP_TYPE_INPUT</code>	Text input chip

Live demo

Live demo[Open in sandbox](#)

|

Live demo[Open in sandbox](#)

|

Live demo[Open in sandbox](#)

|

Live demo[Open in sandbox](#)

|

Live demo[Open in sandbox](#)

|

What's Next

- Learn [Layout & Navigation](#) for containers and scrolling.
- Explore the [Map Widget](#) for interactive maps.
- Try [Incense](#) for rapid prototyping.

Layout & Navigation

Structural widgets that arrange children, handle scrolling, and provide navigation patterns.

Container

```
AromaNode *container = aroma_container_create(root, 0, 0, 800, 480);  
aroma_container_set_layout_mode(container, AROMA_LAYOUT_MODE_FLEX);  
aroma_node_set_flex_direction(container, AROMA_FLEX_COLUMN);
```

Containers are the primary structural widget. They support flexbox, grid, and scrolling.

Live demo [Open in sandbox](#)

I

Scrollable Container

```
aroma_container_set_scrollable(container, true);  
aroma_container_set_scroll_direction(container, AROMA_SCROLL_VERTICAL);
```

Features: - **Velocity tracking** for fling gestures - **Overscroll bounce** with spring-back animation - **Auto content sizing** based on children

Live demo [Open in sandbox](#)

I

ListView

```
AromaNode *list = aroma_ui_listview(root, 0, 0, 300, 400, on_item_select, NULL, font);
aroma_listview_add_item(list, "Item 1", "Secondary text", NULL);
aroma_listview_add_header(list, "Section");
aroma_listview_add_separator(list);
```

Function	Purpose
<code>aroma_listview_add_item</code>	Add a normal row
<code>aroma_listview_add_item_with_icon</code>	Add row with icon
<code>aroma_listview_add_header</code>	Add section header
<code>aroma_listview_add_separator</code>	Add divider line
<code>aroma_listview_clear</code>	Remove all items
<code>aroma_listview_get_selected</code>	Get selected index
<code>aroma_listview_set_font</code>	Change item font
<code>aroma_listview_set_item_hidden</code>	Hide/show individual items (useful for filtering)

ListView is built on a scrollable container. Item heights are calculated dynamically. Use `aroma_listview_set_item_hidden()` to hide items without removing them, then call `aroma_node_invalidate()` to trigger a redraw.

Table

```
AromaNode *table = aroma_table_create(parent, 0, 0, 600, 300, 3);
aroma_table_set_font(table, font);
aroma_table_set_header(table, 0, "Name");
aroma_table_set_header(table, 1, "Size");
aroma_table_set_header(table, 2, "Action");
int row = aroma_table_add_row(table);
aroma_table_set_cell_text(table, row, 0, "Snake");
aroma_table_set_cell_text(table, row, 1, "5.8 KB");
AromaNode *btn = aroma_ui_button(table, "Get", 0, 0, 100, 32,
                                on_get, NULL, font);
aroma_table_set_cell_widget(table, row, 2, btn);
```

Tables render their own grid (headers, zebra rows, selection highlight, dividers) and position embedded cell widgets. They live inside a scroll container for long lists (`aroma_ui_table()` builds that pair for you).

Function	Purpose
<code>aroma_table_add_row</code>	Append a row, returns its index
<code>aroma_table_clear_rows(table, destroy_widgets)</code>	Remove all rows (optionally freeing cell widgets) for rebuilds
<code>aroma_table_set_cell_text</code>	Set a text cell (63 chars max)
<code>aroma_table_set_cell_widget</code>	Embed a button/icon/progress widget in a cell
<code>aroma_table_set_header</code>	Set a column header label
<code>aroma_table_set_header_visible</code>	Hide the header band (e.g. app lists)
<code>aroma_table_set_col_width</code>	Size a column in pixels
<code>aroma_table_set_row_height</code>	Row height in pixels (default 40)
<code>aroma_table_set_selected_row</code> / <code>aroma_table_get_selected_row</code>	Control/read selection (<code>-1</code> = none)
<code>aroma_table_get_row_count</code>	Current row count
<code>aroma_table_set_callback</code>	Row-tap callback with the row index
<code>aroma_table_set_font</code>	Text font (required - nothing draws without it)
<code>aroma_table_destroy</code>	Free rows, cell widgets and the table

Tabs

```
AromaNode *tabs = aroma_ui_tabs(root, 0, 0, 800, 48);
aroma_tabs_add_tab(tabs, "Home", home_content);
aroma_tabs_add_tab(tabs, "Settings", settings_content);
```

Supports animated transitions (`AROMA_ANIM_SLIDE_X`, `AROMA_ANIM_FADE`).

Sidebar

```
AromaNode *sidebar = aroma_ui_sidebar(root, 0, 0, 280, 480);
aroma_sidebar_add_item(sidebar, AROMA_ICON_HOME, "Home", home_content);
```

Responsive: retracts to icon-only mode below a breakpoint.

Menu

```
AromaNode *menu = aroma_menu_create(root, x, y, 200);
aroma_menu_add_item(menu, "Edit", AROMA_MENU_ITEM_NORMAL, on_edit);
aroma_menu_add_item(menu, "Delete", AROMA_MENU_ITEM_DESTRUCTIVE, on_delete);
```

Floating context menu with click-away dismiss.

Live demo [Open in sandbox](#)

I

Live demo

Live demo [Open in sandbox](#)

I

Live demo [Open in sandbox](#)

I

What's Next

- Learn [Input & Controls](#) for interactive elements.
- Explore the [Map Widget](#) for geographic data.
- Try [Incense](#) for declarative UI.

Map Widget

A high-performance, interactive map component supporting OpenStreetMap tiles, markers, and routing.

Quick Start

```
AromaNode *map = aroma_ui_map(root, 0, 0, 700, 400);
aroma_map_set_center(map, 33.8869f, 9.5375f);
aroma_map_add_icon_marker(map, 33.8869f, 9.5375f, 0xFF0000, AROMA_ICON_HOME);
aroma_map_set_route(map, 48.8566, 2.3522, 48.8049, 2.1204, 0xFF35A8FE);
```

Features

- **Tile caching:** LRU cache with 128 tiles by default
- **Cross-platform:** Uses libcurl on native, `emscripten_fetch` on web
- **Markers:** Icon and popup markers with click interaction
- **Routing:** OSRM polyline routing with mutex-safe decoding
- **Physics:** Inertial panning with velocity-based fling

API Reference

Function	Purpose
<code>aroma_map_set_center(lat, lon)</code>	Set map center
<code>aroma_map_set_zoom(level)</code>	Set zoom level (1 to 18)
<code>aroma_map_add_icon_marker(lat, lon, color, icon)</code>	Add icon marker
<code>aroma_map_add_popup_marker(lat, lon, color, text)</code>	Add clickable popup
<code>aroma_map_set_route(lat1, lon1, lat2, lon2, color)</code>	Draw route polyline
<code>aroma_map_remove_route()</code>	Clear current route

Architecture

The map widget runs tile fetching on background worker threads to avoid blocking the UI. It uses spherical mercator projection to convert lat/lon to pixel coordinates and supports pan/zoom gestures with momentum.

What's Next

- Learn [Layout & Navigation](#) for containers and scrolling.
- Explore [Input & Controls](#) for interactive elements.
- Try [Incense](#) for rapid prototyping.

Incense Declarative UI

Incense is the AromaUI language for describing UI in plain text. Write widgets, click Run, and see the result in the browser at once.

Open the Sandbox

[Launch Incense Sandbox](#)

Edit code on the left, click **Run**, and see the preview update at once with WebAssembly.

Syntax

A container with `layout: flex` positions its children itself. Inside a flex container, a child keeps its size but its `x` and `y` are set by the flex row or column, so explicit `x` and `y` on direct children are ignored. Use a plain container (no `layout`) when you want exact `x` and `y` positions.

```
Window {  
  width: 320  
  height: 480  
  title: "My App"  
  
  Container {  
    x: 0  
    y: 0  
    width: 320  
    height: 480  
    layout: flex  
    direction: column  
  
    Label {  
      text: "Hello"  
      style: large  
      color: #333333  
    }  
  
    Button {  
      text: "Click Me"  
      x: 20  
      y: 100  
      width: 120  
      height: 40  
      on_click: "handle_click"  
    }  
  }  
}
```

Supported Widgets

Widget	Key Properties	Children
Button	text, x, y, width, height, on_click	-
Label	text, x, y, style, color	-
Container	x, y, width, height, layout, direction	Any widget
Card	x, y, width, height, type	-
Checkbox	label, x, y, checked, on_change	-
Switch	x, y, width, height, value, on_change	-
Slider	x, y, width, height, min, max, value, on_change	-
Textbox	x, y, width, height, placeholder, on_change	-
ProgressBar	x, y, width, height, value	-
Divider	x, y, length, orientation	-
IconButton	x, y, width, height, icon, on_click	-
Icon	x, y, size, text, color	-
Image	x, y, width, height, src	-
Dropdown	x, y, width, height, on_change	Option

Widget	Key Properties	Children
RadioButton	label, x, y, width, height, group, checked, on_click	-
Tabs	x, y, width, height, on_change	Tab
Sidebar	x, y, width, height	Item
Menu	x, y, width	MenuItem
Chip	x, y, label, icon, type, selected	-
Tooltip	text, x, y, position	-
GIF	x, y, width, height, src, autoplay	-
Loading	x, y, radius, thickness	-
Gauge	x, y, width, height, value	-
Canvas	x, y, width, height	-
DebugOverlay	x, y, width, height, visible	-
Map	x, y, width, height, lat, lon, zoom	-
Snackbar	message, duration, action, on_click	-
ListView	x, y, width, height, on_select	ListItem, Header, Separator
Dialog		-

Widget	Key Properties	Children
	<code>title</code> , <code>message</code> , <code>width</code> , <code>height</code> , <code>type</code>	
<code>ScrollView</code>	<code>x</code> , <code>y</code> , <code>width</code> , <code>height</code> , <code>direction</code>	Any widget
<code>Table</code>	<code>x</code> , <code>y</code> , <code>width</code> , <code>height</code>	<code>Column</code> , <code>Row</code> , <code>HeaderCell</code>

Property Types

Type	Example	Notes
Integer	<code>x: 20</code>	Whole numbers
Float	<code>value: 0.5</code>	Decimal numbers
Boolean	<code>checked: true</code>	<code>true</code> or <code>false</code>
Color	<code>color: #FF0000</code>	Hex color
String	<code>text: "Hello"</code>	Double-quoted
Enum	<code>style: large</code>	Unquoted keyword
Object	<code>Container { ... }</code>	Nested widget

Special Features

- **Callbacks:** Reference C-registered callbacks: `on_click: "handle_click"`
- **Embeds:** Include other files: `@embed "shared.aroma"`
- **Comments:** Single-line with `//`

- **List children:** `ListItem { text: "... " secondary: "... " }` for `ListView`
- **Navigation:** Use `visible: 0/1` to show/hide pages and `on_click: "back"` for back navigation
- **Icons:** Material icon names: `AROMA_ICON_*` constants for `IconButton` and `Icon` widgets
- **Virtual Keyboard:** Textbox widgets automatically show a responsive virtual keyboard when focused on Emscripten/WebAssembly builds. The keyboard anchors at the bottom of the canvas, keys scale to fit screen width, and labels abbreviate when space is limited. Enable it programmatically with `aroma_textbox_enable_virtual_keyboard(root, true)`.

Known Limitations

1. No arrays/lists as property values - use child objects instead
2. No control flow - all widgets render every frame
3. No data binding - properties are static literals
4. No expressions - cannot compute values from other values
5. No component system - no user-defined widgets yet
6. No declarative animations
7. Limited event types - `on_click`, `on_change`, `on_select`, `on_submit`
8. Single window per file
9. No string interpolation
10. Hard limits: 64 children per parent, 64 properties per widget

Emscripten Build

For WebAssembly builds, the following must be included in your `CMakeLists.txt`:

- `aroma_textbox.c` must be added to the executable's source list (it is not part of the default library link on Emscripten)
- `aroma_textbox_enable_virtual_keyboard` and related functions must be listed in `EXPORTED_FUNCTIONS`

- `aroma_textbox.c` uses `__EMSCRIPTEN__` guards for web-only VK code paths

Example CMakeLists.txt configuration:

```
add_executable(my_app main.c ../../src/widgets/aroma_textbox.c)

target_link_options(my_app PRIVATE
    "-sEXPORTED_FUNCTIONS=['_main','_malloc','_free',...,'_aroma_textbox_enable_virtual_key']"
    "-sEXPORTED_RUNTIME_METHODS=['ccall','cwrap','FS','addRunDependency','removeRunDependency']"
)
```

Sample: Wi-Fi Connect Page

The Incense Sandbox includes a sample iOS-style Wi-Fi connection page demonstrating multi-page navigation with text input:

```
```aroma Container { id: "page_wifi_connect" x: 0 y: 0 width: 320 height: 480 layout: flex
direction: column visible: 0
```

```

Container {
 x: 0
 y: 0
 width: 320
 height: 56
 color: #121212

 IconButton {
 x: 8
 y: 8
 width: 40
 height: 40
 icon: "AROMA_ICON_ARROW_BACK"
 variant: standard
 on_click: "back"
 }

 Label {
 text: "Wi-Fi"
 style: large
 color: #FFFFFF
 y: 20
 x: 60
 }
}

Container {
 x: 20
 y: 70
 width: 280
 height: 340

 Textbox {
 text: ""
 placeholder: "Network Name (SSID)"
 x: 20
 y: 30
 width: 240
 height: 44
 }

 Textbox {

```

```
 text: ""
 placeholder: "Password"
 x: 20
 y: 90
 width: 240
 height: 44
 }

 Button {
 text: "Connect to Network"
 x: 40
 y: 170
 width: 200
 height: 44
 on_click: "wifi_connect"
 }
}
```

```
}
```

## What's Next

- Learn the C APIs in [Widget Library](#).
- Read [Core Framework](#) to see how Incense maps to `AromaNode`.
- Try the [live sandbox](#).

# Project Creation

---

`aroma create` makes a new cross platform project in seconds. Run it from the AromaUI repo root.

## Usage

```
python3 bin/aroma create MyApp
cd MyApp
```

Type a project name. Use a plain name with no spaces, such as `MyApp`.

## What you get

```
MyApp/
 src/main.c # Entry point
 CMakeLists.txt # Build config for Linux and Web
 android/ # Android project
 app/
 src/main/
 AndroidManifest.xml
 java/ # JNI bridge
 cpp/ # Native build config
```

## Starter code

`src/main.c` is a full working app. It opens a window, shows text, and runs the main loop:

```
int main(void) {
 aroma_ui_init();
 AromaWindow *window = aroma_ui_create_window("Hello World", 400, 600);
 // ... build UI ...
 while (aroma_ui_is_running()) {
 aroma_ui_process_events();
 aroma_ui_render(window);
 }
 aroma_ui_shutdown();
 return 0;
}
```

On Android, the template also wires `android_main()` to the NDK app state. You do not need to change this to start.

## Build it

```
python3 ../bin/aroma build linux
python3 ../bin/aroma run linux
```

Run Android and Web builds from the same folder. See [Building and Deployment](#).

## What's next

- Learn to [build for all platforms](#).
- Explore the [Widget Library](#) to build your UI.
- Try [Incense](#) to prototype screens in your browser.

# Building & Deployment

---

AromaUI builds for three targets: Linux, Android, and Web. The `aroma` CLI wraps each toolchain. Run all commands inside your project folder.

## Commands

Command	What it does
<code>python3 bin/aroma build linux</code>	Compile with CMake and Make
<code>python3 bin/aroma build android</code>	Build a debug APK with Gradle
<code>python3 bin/aroma build android --release</code>	Build a signed release APK or AAB
<code>python3 bin/aroma build web</code>	Compile to WASM with Emscripten
<code>python3 bin/aroma run linux</code>	Build and launch on Linux
<code>python3 bin/aroma run android --emu</code>	Install and launch on an emulator
<code>python3 bin/aroma run web</code>	Build for Web

Check your setup first:

```
python3 bin/aroma doctor
```

## Linux build

```
python3 bin/aroma build linux
python3 bin/aroma run linux
```

The same steps by hand:

```
mkdir build && cd build
cmake ..
make -j$(nproc)
./MyApp
```

The binary name matches your project folder. If `run` cannot find it, open `build/` and launch the binary directly.

## Android build

You need JDK 17, an Android SDK platform, build tools, NDK, and CMake. The build asks you to pick versions the first time and saves them.

```
Debug APK
python3 bin/aroma build android

Signed release APK
python3 bin/aroma build android --release

App Bundle for Google Play
python3 bin/aroma build android --aab
```

Run on a device or emulator:

```
python3 bin/aroma run android --emu
```

For release builds, set up your keystore in the Android project first. Gradle signs the output.

## Web build

You need the Emscripten SDK under `vendors/emscripten` or `~/.aroma/vendors/emscripten`.

```
python3 bin/aroma build web
python3 -m http.server 8080 --directory build_web
```

Open `http://localhost:8080` in your browser. Output goes to `build_web/`.

## Build flow

```
aroma build <target>
 linux: CMake, then Make, then a native binary
 android: Gradle, then an APK or AAB
 web: Emscripten with CMake, then WASM plus JS
```

## Common errors

- `Emscripten SDK not found`: install the SDK in one of the paths above, then retry.
- `No Android SDK platforms found`: install a platform with the SDK Manager and set `ANDROID_HOME`.
- `Not an Aroma project (no android/ directory)`: run the command inside a folder made by `aroma create`.
- `Executable not found. Run 'aroma build linux' first`: build before you run.

## What's next

- Learn [Project Creation](#) to scaffold a new app.
- Explore the [Widget Library](#) to build your UI.
- Check [Testing and Quality](#) for CI setup.

# SDK Downloads

---

## SDK Downloads

Get the latest AromaUI release for your platform



Fetching latest release from GitHub...

# Graphics Backends

---

The Graphics Backend subsystem in AromaUI provides a hardware-agnostic interface for rendering primitives, text, and images. It is managed by the **Backend Abstraction Layer (ABI)**, which routes high-level draw commands to one of three specialized implementations: **GL ES3**, **Vulkan**, or **TFT\_eSPI**.

## Backend Architecture Overview

The `AromaGraphicsInterface` structure defines the contract that every graphics backend must implement. This includes lifecycle management, primitive drawing (rectangles, arcs), text rendering via FreeType, and image loading using `stb_image` or `NanoSVG`.

### Graphics Interface Contract

Function Category	Key Functions
<b>Lifecycle</b>	<code>setup_shared_window_resources</code> , <code>setup_separate_window_resources</code> , <code>shutdown</code>
<b>Primitives</b>	<code>fill_rectangle</code> , <code>draw_hollow_rectangle</code> , <code>draw_arc</code>
<b>Text</b>	<code>render_text</code> , <code>measure_text</code>
<b>Images</b>	<code>load_image</code> , <code>load_image_from_memory</code> , <code>draw_image</code>
<b>State</b>	<code>graphics_set_clip</code> , <code>graphics_clear_clip</code> , <code>notify_dirty_region</code>

**Sources:** [src/backends/graphics/aroma\\_graphics\\_interface.h15-107](src/backends/graphics/aroma_graphics_interface.h15-107)

---

## GL ES3 Backend

The GL ES3 backend is the primary renderer for Desktop (Linux/GLFW) and Android targets. It utilizes modern OpenGL ES 3.0 features to achieve high-performance UI rendering through batching and specialized shaders.

### Implementation Details

- **Shape Batching:** To minimize draw calls, the backend uses a `ShapeBatch` structure [src/backends/graphics/aroma\\_graphics\\_gles3.c50-56](#) Quads are collected into a vertex buffer (VBO) and flushed in batches of up to 256 quads [src/backends/graphics/aroma\\_graphics\\_gles3.c24-26](#)
- **Rounded-Rect SDF Shaders:** Rectangles are rendered using a fragment shader (`rectangle_fragment_shader`) that calculates Signed Distance Fields (SDF) to provide perfectly anti-aliased rounded corners and borders without additional geometry [src/backends/graphics/aroma\\_graphics\\_gles3.c227-230](#)
- **LRU Font Cache:** Each window maintains a `CachedFontRenderer` [src/backends/graphics/aroma\\_graphics\\_gles3.c59-64](#) If the cache exceeds `MAX_FONT_CACHE_PER_WINDOW` (16), the Least Recently Used (LRU) font is evicted to save GPU memory [src/backends/graphics/aroma\\_graphics\\_gles3.c114-144](#)
- **Image Loading:** Integrates `stb_image` for raster formats and `NanoSVG` for vector assets, converting them into GL textures [src/backends/graphics/aroma\\_graphics\\_gles3.c12-18](#)

## GLES3 Entity Mapping

📄 flowchart LR

```

flowchart LR
 subgraph Buffers
 SVB0["shape_vbo"]
 TVB0["text_vbo"]
 SVA0["shape_vao"]
 end
 subgraph Shaders
 SP["shape_program"]
 TP["text_program"]
 end
 subgraph subGraph0 ["GLES3 Context"]
 CTX["AromaGLS3Context (ctx)"]
 SB["ShapeBatch"]
 WR["WindowResources (windows#91;#93;)"]
 end
 CTX --> SB
 CTX --> WR
 WR --> SVA0
 WR --> TP
 SB --> SVB0
 SB --> SP

```

**Sources:** [src/backends/graphics/aroma\\_graphics\\_gles3.c28-95](src/backends/graphics/aroma_graphics_gles3.c28-95)

## Vulkan Backend

The Vulkan backend provides a high-efficiency alternative for Android and Linux platforms, specifically targeting low-overhead draw call submission and explicit resource management.

## Key Features

- **Surface Management:** Uses `vkCreateAndroidSurfaceKHR` (on Android) or KHR surface extensions provided by the platform interface to bind to native windows [src/backends/graphics/aroma\\_graphics\\_vulkan.c163-174](#)
- **Command Buffering:** Implements a double-buffered frame approach using `VK_MAX_FRAMES_IN_FLIGHT` (2) to allow the CPU to record commands for the next frame while the GPU processes the current one [src/backends/graphics/utils/helpers\\_vulkan.h21](#)
- **Push Constants:** Uses `VkWhiteTexture` and push constants for immediate updates to projection matrices and shape properties (radius, border width) without updating descriptor sets [src/backends/graphics/utils/helpers\\_vulkan.h104-114](#)
- **Text Pipeline:** Uses a dedicated `VulkanTextRenderer` that manages a glyph atlas and specific pipelines for alpha-blended text [src/backends/graphics/aroma\\_graphics\\_vulkan.c23-29](#)

**Sources:** [src/backends/graphics/aroma\\_graphics\\_vulkan.c51-70](#) [src/backends/graphics/utils/helpers\\_vulkan.h62-140](#)

---

## TFT\_eSPI Backend (Embedded)

Designed for resource-constrained microcontrollers (e.g., ESP32), this backend interacts with the `TFT_eSPI` library. It emphasizes memory efficiency through tiling and smart flushing.

### Tiled Rendering and Double Buffering

Since many embedded targets lack enough RAM for a full-screen frame buffer, AromaUI employs a **Tiled Rendering** strategy:

1. The screen is divided into vertical tiles of height `TILE_H` (typically 100px) [src/backends/platforms/aroma\\_platform\\_tft\\_espi.cpp21](#)

2. A `TFT_eSprite` is used as a temporary back-buffer for a single tile [src/backends/platforms/aroma\\_platform\\_tft\\_espi.cpp62-68](#)
3. The `call_flush_function_ptr` iterates through dirty tiles, renders the `DrawList` clipped to that tile's bounds, and pushes the sprite to the physical display via SPI [src/backends/platforms/aroma\\_platform\\_tft\\_espi.cpp169-200](#)

## TFT\_eSPI Data Flow

sequenceDiagram

```
sequenceDiagram
 participant SG as Scene Graph
 participant DL as AromaDrawList
 participant ABI as Backend ABI
 participant TFT as TFT_eSPI Backend
 participant SPR as TFT_eSprite
 SG->>DL: Record Commands (Rect, Text)
 DL->>ABI: aroma_drawlist_flush()
 ABI->>TFT: call_flush_function_ptr()
 TFT->>SPR: fillRect(Clear Color)
 TFT->>DL: Execute commands with ClipRect
 DL->>SPR: drawRoundRect / print
 TFT->>SPR: pushSprite(0, tile_y)
```

**Sources:**[src/backends/platforms/aroma\\_platform\\_tft\\_espi.cpp28-38](#)[src/backends/graphics/aroma\\_graphics\\_tft\\_espi.cpp103-113](#)

## The DrawList System

The `AromaDrawList` acts as a command buffer between the UI widgets and the graphics backends. This allows for deferred rendering, Z-index sorting, and the "Smart Flush" mechanism used by the TFT backend.

## Command Buffering

Widgets do not call graphics functions directly. Instead, they record `AromaDrawCmd` entries into an active list:

- `aroma_drawlist_cmd_fill_rect` [src/core/aroma\\_drawlist.c174-189](#)
- `aroma_drawlist_cmd_text` [src/core/aroma\\_drawlist.c226-241](#)
- `aroma_drawlist_cmd_image` [src/core/aroma\\_drawlist.c243-256](#)

## Smart Flush

The `aroma_drawlist_smart_flush` function is critical for tiled rendering. It accepts a clipping rectangle and only executes commands that intersect with that region, significantly reducing processing time for partial updates [include/aroma\\_drawlist.h186](#)

**Sources:**[src/core/aroma\\_drawlist.c28-85](#)[include/aroma\\_drawlist.h37-46](#)

# Platform Backends

---

Platform backends in AromaUI serve as the hardware and operating system abstraction layer. They implement the `AromaPlatformInterface` defined in [src/backends/platforms/aroma\\_platform\\_interface.h35-265](#) providing standardized hooks for window management, input capture, and system-specific services (e.g., Android Bluetooth or JNI bridges). This architecture allows the Core Framework to remain platform-agnostic while supporting diverse targets ranging from embedded ESP32 displays to desktop Linux and mobile Android environments.

## Architecture and Abstraction

The `AromaPlatformInterface` acts as a contract that each backend must fulfill. During initialization, the `AromaBackendABI` routes calls to the active platform implementation [src/core/aroma\\_ui\\_impl.c170-181](#)

## Platform Interface Lifecycle

1. **Initialize:** Sets up the native windowing system or hardware drivers [src/backends/platforms/aroma\\_platform\\_interface.h43](#)
2. **Window Creation:** Returns a `size_t` window ID used for context management [src/backends/platforms/aroma\\_platform\\_interface.h62-66](#)
3. **Event Loop:** A blocking or non-blocking call that processes native OS messages and translates them into `AromaEvent` objects [src/backends/platforms/aroma\\_platform\\_interface.h103](#)
4. **Shutdown:** Releases native resources [src/backends/platforms/aroma\\_platform\\_interface.h48](#)

## Platform-to-Code Mapping

The following diagram maps high-level platform concepts to the specific C structures and functions that implement them.

## Platform Backend Entity Mapping

📄 flowchart LR

```

flowchart LR
 subgraph subGraph2 ["Interface Definitions"]
 API["AromaPlatformInterface"]
 ABI["AromaBackendABI"]
 end
 subgraph subGraph1 ["AromaUI Backend Implementation"]
 GLPS["aroma_platform_glps.c"]
 GLFW["aroma_platform_glfw.c"]
 ANDROID["aroma_platform_android.c"]
 TFT["aroma_platform_tft_espi.cpp"]
 end
 subgraph subGraph0 ["Native OS Layer"]
 X11["X11/Wayland"]
 WIN32["Win32 API"]
 AND_APP["android_app struct"]
 ESP_DRV["TFT_eSPI Driver"]
 end
 X11 --> GLPS
 WIN32 --> GLFW
 AND_APP --> ANDROID
 ESP_DRV --> TFT
 GLPS -->|"implements"| API
 GLFW -->|"implements"| API
 ANDROID -->|"implements"| API
 TFT -->|"implements"| API
 API -->|"exposed via"| ABI

```

Sources: [src/backends/platforms/aroma\\_platform\\_interface.h35-265](#)[src/backends/platforms/aroma\\_platform\\_glps.c31-38](#)[src/backends/platforms/aroma\\_platform\\_android.c50-58](#)

## GLPS (Linux Desktop Default)

The GLPS backend is the primary target for Linux and desktop development. It utilizes the `glps_WindowManager` to handle X11 or Wayland surfaces [src/backends/platforms/aroma\\_platform\\_glps.c200-205](src/backends/platforms/aroma_platform_glps.c200-205)

### Input Translation

GLPS captures native events and maps them to the AromaUI event system:

- **Mouse:** `glps_mouse_move_callback` and `glps_mouse_click_callback` calculate deltas and trigger `aroma_event_queue` [src/backends/platforms/aroma\\_platform\\_glps.c86-114](src/backends/platforms/aroma_platform_glps.c86-114)
- **Keyboard:** Translates X11 keycodes to ASCII/UTF-8 values and handles modifiers like `AROMA_KEY_MOD_CAPSLOCK` [src/backends/platforms/aroma\\_platform\\_glps.c116-173](src/backends/platforms/aroma_platform_glps.c116-173)
- **Scrolling:** Maps vertical/horizontal scroll axes to `EVENT_TYPE_MOUSE_SCROLL` [src/backends/platforms/aroma\\_platform\\_glps.c176-196](src/backends/platforms/aroma_platform_glps.c176-196)

Sources: [src/backends/platforms/aroma\\_platform\\_glps.c1-243](src/backends/platforms/aroma_platform_glps.c1-243)

---

## GLFW (Cross-Platform & Headless)

The GLFW backend provides a portable implementation for Windows, macOS, and Linux. It supports both standard windowed modes and a specialized "surfaceless" mode for headless rendering [src/backends/platforms/aroma\\_platform\\_glfw.c38-51](src/backends/platforms/aroma_platform_glfw.c38-51)

## Surfaceless/EGL Path

For environments without a display server (e.g., GStreamer pipelines), the backend initializes an EGL context using `EGL_KHR_surfaceless_context` [src/backends/platforms/aroma\\_platform\\_glfw.c63-182](#)

- **Context:** Uses `eglGetDisplay(EGL_DEFAULT_DISPLAY)` and `eglCreateContext` with `EGL_PBUFFER_BIT` [src/backends/platforms/aroma\\_platform\\_glfw.c103-157](#)
- **Use Case:** Integrated in `examples/gstreamer_example/ev_cluster.c` to render UI frames directly into shared memory (`/aroma_frame_shm`) for video encoding [examples/gstreamer\\_example/ev\\_cluster.c1-18](#)

Sources: [src/backends/platforms/aroma\\_platform\\_glfw.c1-219](#)[examples/gstreamer\\_example/ev\\_cluster.c1-102](#)

---

## Android Backend

The Android backend manages the complex lifecycle of an `ANativeActivity` and coordinates with the Java-side `AromaHelper` via JNI [src/backends/platforms/aroma\\_platform\\_android.c1-30](#)

### Frame Scheduling with AChoreographer

Unlike desktop backends that might poll, Android uses the system `AChoreographer` to sync rendering with the display refresh rate (VSYNC) [src/backends/platforms/aroma\\_platform\\_android.c80-85](#)

- **choreographer\_callback**: Triggered by the OS; it calls `update_surface_size` and executes the registered `window_update_callback` [src/backends/platforms/aroma\\_platform\\_android.c86-114](#)
- **request\_frame**: Posts a new callback to the choreographer only when a redraw is needed [src/backends/platforms/aroma\\_platform\\_android.c116-124](#)

## Data Flow: JNI and System Services

The backend maintains an `AromaHelperCache` of JNI method IDs to call into Java for system services [src/backends/platforms/aroma\\_platform\\_android.c133-171](src/backends/platforms/aroma_platform_android.c133-171)

Feature	Implementation Mechanism	Code Reference
<b>Permissions</b>	<code>platform-&gt;android_check_permission</code> via JNI	<a href="include/aroma_android.h89-95">include/aroma_android.h89-95</a>
<b>Bluetooth</b>	<code>android_bt_scan</code> calling Java <code>bt_scan</code>	<a href="include/aroma_android.h199-204">include/aroma_android.h199-204</a>
<b>Haptics</b>	<code>android_vibrate</code> calling Java <code>vibrator</code>	<a href="include/aroma_android.h136-144">include/aroma_android.h136-144</a>
<b>Lifecycle</b>	<code>android_native_app_glue</code> events (Focus/Pause)	<a href="src/backends/platforms/aroma_platform_android.c4-7">src/backends/platforms/aroma_platform_android.c4-7</a>

### Android Backend Internal Flow

sequenceDiagram

```
sequenceDiagram
 participant OS as Android OS (VSYNC)
 participant AC as AChoreographer
 participant PB as aroma_platform_android.c
 participant UI as AromaUI Core
 participant JNI as AromaHelper.java
 OS->>AC: VSYNC Signal
 AC->>PB: choreographer_callback()
 PB->>UI: window_update_callback()
 UI->>PB: swap_buffers()
 PB->>JNI: (Optional) aroma_android_vibrate() via JNI
```

Sources: [src/backends/platforms/aroma\\_platform\\_android.c86-114](#)[include/aroma\\_android.h136-144](#)

---

## TFT\_eSPI and Emscripten

### TFT\_eSPI (Embedded)

The TFT\_eSPI backend targets ESP32 and similar microcontrollers. It is unique in its support for **Tiled Rendering**.

- **Dirty Regions:** It tracks dirty tiles via `tft_mark_tiles_dirty` to minimize SPI bus traffic [src/backends/platforms/aroma\\_platform\\_interface.h135](#)
- **Flush Mechanism:** The `call_flush_function_ptr` allows the Core to push `AromaDrawList` contents to the `TFT_eSprite` buffer [src/backends/platforms/aroma\\_platform\\_interface.h122-130](#)

### Emscripten (Web)

For WebAssembly targets, the backend utilizes Emscripten's browser hooks.

- **DPI Scaling:** Uses `aroma_emscripten_device_pixel_ratio` to scale the UI for high-density (Retina) displays [src/core/aroma\\_ui\\_impl.c84-88](#)
- **Main Loop:** Employs `emscripten_set_main_loop` (abstracted in the platform's `run_event_loop`) to prevent blocking the browser's UI thread.

Sources: [src/backends/platforms/aroma\\_platform\\_interface.h110-141](#)[src/core/aroma\\_ui\\_impl.c84-102](#)

# Font System

---

AromaUI's font system loads, manages, and renders typography across Linux, Android, Web, and embedded targets.

## Quick Start

```
// Load from bundled Ubuntu font
AromaFont *font = aroma_font_create_from_memory(
 aroma_ubuntu_ttf, aroma_ubuntu_ttf_len, 24);

// Or from disk
AromaFont *font = aroma_font_create("/path/to/font.ttf", 24);

// Measure text
int w = aroma_font_get_text_width(font, "Hello", 12);
int h = aroma_font_get_line_height(font);
```

## Architecture

- **Core** ( `aroma_font.c` ) - platform-agnostic font loading and metrics
- **FreeType** - vector rasterization on Linux/Android/Web
- **GL ES3 backend** - GPU glyph cache with LRU eviction
- **ESP32/TFT** - bypasses FreeType, uses built-in bitmap fonts

## Key APIs

Function	Purpose
<code>aroma_font_create(path, size)</code>	Load font from filesystem
<code>aroma_font_create_from_memory(data, len, size)</code>	Load from memory buffer
<code>aroma_font_destroy(font)</code>	Release font resources
<code>aroma_font_get_text_width(font, text, max_width)</code>	Measure text width
<code>aroma_font_get_line_height(font)</code>	Get recommended line spacing
<code>aroma_font_get_ascender(font)</code>	Distance from baseline to top
<code>aroma_font_get_descender(font)</code>	Distance from baseline to bottom

## Glyph Caching (GLES3)

The GLES3 backend maintains a per-window glyph cache. Characters 32 to 127 (ASCII) are pre-loaded on font initialization. Extended characters are loaded on demand and cached as GL textures.

- Cache limit: 16 fonts per window (LRU eviction)
- Texture format: `GL_RED` (single channel) for memory efficiency

## DP/SP Scaling

AromaUI supports density-independent pixels (DP) for layouts and scale-independent pixels (SP) for fonts. On Android, SP values are converted to physical pixels based on the device's display metrics.

```
#define FONT_TITLE_SP 36
int px = sp(FONT_TITLE_SP);
AromaFont *font = aroma_font_create_from_memory(ttf, len, px);
```

## Platform Specifics

- **ESP32:** Uses fixed bitmap fonts ( `FreeSans12pt7b` , `GLCD` ). No FreeType dependency.
- **Web/WASM:** Uses Emscripten's file system for font loading, or memory buffers for portability.
- **Android:** Bridges to system font services via JNI when needed.

## What's Next

- Explore [Theming](#) for typography configuration.
- Learn [Rendering](#) for text draw optimization.
- Check [Testing & Quality](#) for font rendering tests.

# Testing & Quality

---

The AromaUI test suite provides a comprehensive set of unit and integration tests designed to ensure the stability of core UI primitives, memory management, and event propagation. The testing architecture is decoupled from any specific platform backend, allowing tests to run in a headless environment (typically Linux) to validate logic without requiring a GPU or display.

## Test Suite Architecture

The test suite is organized into modular components targeting specific subsystems. The entry point is `test_main.c`, which orchestrates the execution of different test modules and aggregates results.

## Test Execution Flow

Title: AromaUI Test Execution Pipeline

## flowchart LR

```

flowchart LR
 subgraph subGraph1 ["Subsystem Modules"]
 SLAB_MOD["test_aroma_slab_alloc.c"]
 NODE_MOD["test_aroma_node.c"]
 EVENT_MOD["test_aroma_event_system.c"]
 end
 subgraph subGraph0 ["Test Runner #91;test_main.c#93;"]
 START["main()"]
 SLAB["run_slab_allocator_tests()"]
 NODE["run_node_tests()"]
 EVENT["run_event_tests()"]
 SUMMARY["Print Summary & Exit"]
 end
 START --> SLAB
 SLAB --> NODE
 NODE --> EVENT
 EVENT --> SUMMARY
 SLAB -. -> SLAB_MOD
 NODE -. -> NODE_MOD
 EVENT -. -> EVENT_MOD

```

Sources: [tests/test\\_main.c84-100](#)[tests/core/test\\_aroma\\_slab\\_alloc.c232-242](#)

## Memory Allocator Tests ( `test_aroma_slab_alloc.c` )

These tests validate the `AromaSlabAllocator`, which is critical for deterministic memory performance on embedded targets. The tests focus on the multi-cache slab system used for both internal `AromaNode` structures and variable-sized widget data.

Key functions tested:

- **Initialization:** `test_memory_system_init` ensures `aroma_memory_system_init` and `aroma_memory_system_destroy` correctly manage the global allocator state [tests/core/test\\_aroma\\_slab\\_alloc.c37-45](#)
- **Widget Allocation:** `test_widget_allocation` verifies that `aroma_widget_alloc` provides memory for different sizes and that

`aroma_widget_free` allows for subsequent re-allocation [tests/core/test\\_aroma\\_slab\\_alloc.c47-74](#)

- **Stress Testing:** `test_widget_allocation_stress` performs 100 interleaved allocations and deallocations of varying sizes to detect fragmentation or pool exhaustion [tests/core/test\\_aroma\\_slab\\_alloc.c76-110](#)
- **Node Pooling:** `test_node_allocation` specifically targets the fixed-size pool used for `AromaNode` structs via `__slab_pool_alloc` [tests/core/test\\_aroma\\_slab\\_alloc.c112-140](#)

Sources: [tests/core/test\\_aroma\\_slab\\_alloc.c1-242](#)

## Scene Graph Tests ( `test_aroma_node.c` )

The node system tests verify the integrity of the UI tree, including parent-child relationship management, ID generation, and the cleanup of complex hierarchies.

### Node Management Logic

Title: Node Lifecycle and Hierarchy Validation

## flowchart LR

```

flowchart LR
 subgraph subGraph1 ["Test Scenarios"]
 T1["test_node_id_generation"]
 T2["test_create_root_node"]
 T3["test_node_hierarchy"]
 T4["test_remove_child_node"]
 end
 subgraph subGraph0 ["Code Entities"]
 GEN["__generate_node_id()"]
 CREATE["__create_node()"]
 ADD["__add_child_node()"]
 REM["__remove_child_node()"]
 DEST["__destroy_node()"]
 end
 T1 --> GEN
 T2 --> CREATE
 T3 --> ADD
 T4 --> REM
 REM --> DEST

```

Key validation logic:

- **ID Uniqueness:** `test_node_id_generation` asserts that every call to `__generate_node_id` returns a strictly increasing, unique identifier [tests/core/test\\_aroma\\_node.c59-73](#)
- **Hierarchy Integrity:** `test_node_hierarchy` builds a tree (Root -> Container -> Button) and asserts that `child_count` and `parent_node` pointers are correctly maintained across levels [tests/core/test\\_aroma\\_node.c140-160](#)
- **Partial Tree Removal:** `test_remove_child_node` verifies that removing a node from the middle of a child array correctly shifts the remaining siblings to maintain array density [tests/core/test\\_aroma\\_node.c162-196](#)

Sources: [tests/core/test\\_aroma\\_node.c59-196](#)

## Event System Tests ( `test_aroma_event_system.c` )

These tests simulate user interaction to verify hit-testing, subscription priorities, and the event bubbling mechanism.

### Event Propagation Model

Title: Event Dispatch and Bubbling Test Flow

sequenceDiagram

```
sequenceDiagram
 participant T as test_event_bubbling
 participant E as aroma_event_dispatch
 participant B as Button Node
 participant C as Container Node
 participant R as Root Node
 T->>R: aroma_event_subscribe(RootID)
 T->>E: aroma_event_create(CLICK, ButtonID)
 E->>B: Process Local Handlers
 B->>C: Bubble Up
 C->>R: Bubble Up
 R->>T: handler_call_count++
```

Key behaviors verified:

- **Subscription:** `test_event_subscription_and_dispatch` ensures that `aroma_event_subscribe` correctly registers a callback and that `aroma_event_dispatch` triggers it when the target ID matches [tests/core/test\\_aroma\\_event\\_system.c95-123](#)
- **Bubbling:** `test_event_bubbling` verifies that an event targeting a leaf node (Button) propagates up to the Root if not consumed [tests/core/test\\_aroma\\_event\\_system.c125-159](#)
- **Consumption:** `test_event_consumption_stops_bubbling` validates that calling `aroma_event_consume` inside a high-priority handler prevents lower-priority or parent-node handlers from receiving the event [tests/core/test\\_aroma\\_event\\_system.c161-199](#)

- **Queueing:** `test_event_queue_processing` tests the asynchronous event path where events are added via `aroma_event_queue` and processed later during `aroma_event_process_queue` [tests/core/test\\_aroma\\_event\\_system.c201-231](#)

Sources: [tests/core/test\\_aroma\\_event\\_system.c95-231](#)

## Build Configuration

The test suite is integrated into the CMake build system, allowing for automated testing and memory sanitization.

### CMake Integration ( `tests/CMakeLists.txt` )

The `aroma_tests` executable links against the core `aroma` library and includes the project headers.

Feature	Implementation
<b>Target Name</b>	<code>aroma_tests</code> <a href="#">tests/CMakeLists.txt1</a>
<b>Address Sanitizer</b>	Enabled via <code>ENABLE_ASAN</code> , adding <code>-fsanitize=address</code> to catch memory leaks and buffer overflows <a href="#">tests/CMakeLists.txt16-22</a>
<b>Test Registration</b>	Uses <code>add_test</code> to integrate with CTest <a href="#">tests/CMakeLists.txt24</a>
<b>Includes</b>	Includes both <code>include/</code> and the local <code>tests/</code> directory for private test headers <a href="#">tests/CMakeLists.txt11-14</a>

Sources: [tests/CMakeLists.txt1-24](#)

# Glossary

---

This page provides definitions for codebase-specific terms, abbreviations, and domain concepts used throughout the AromaUI framework. It serves as a technical reference for onboarding engineers to understand the internal nomenclature and implementation pointers.

## Core Concepts

### AromaNode

The fundamental building block of the AromaUI scene graph. Every UI element (windows, buttons, containers) is an `AromaNode`. It manages the hierarchical parent-child relationship (limited to 64 children for performance) and maintains spatial properties like `AromaRect`.

- **Implementation:** [include/aroma\\_node.h1-50](#)
- **Lifecycle:** Initialized via `__node_system_init` [src/core/aroma\\_ui\\_impl.c165](#)

### AromaBackendABI

The "Application Binary Interface" proxy layer that decouples the core framework from specific graphics and platform implementations. It routes generic draw calls (e.g., `draw_rect`) to the active backend (GLES3, Vulkan, or TFT\_eSPI).

- **Implementation:** [src/backends/aroma\\_abi.c1-100](#)
- **Access:** Accessed via the global `aroma_backend_abi` struct [src/core/aroma\\_ui\\_impl.c33](#)

## DrawList

A command-buffer system used for deferred rendering. Instead of immediate execution, UI nodes record drawing commands into an `AromaDrawList`. This allows for Z-index sorting, frustum culling, and batching before the final flush to the GPU.

- **Implementation:** [src/core/aroma\\_drawlist.c1-150](#)
- **Usage:** Collected during the frame update via `collect_draw_tasks` [src/core/aroma\\_ui\\_impl.c71-73](#)

## Dirty-Region Tracking

An optimization technique where only portions of the screen that have changed are re-rendered. Nodes are marked "dirty" via `aroma_node_invalidate`, adding them to a global dirty list.

- **Implementation:** [src/core/aroma\\_node.c120-150](#)
  - **Trigger:** `aroma_node_invalidate(g_main_window)` [src/core/aroma\\_ui\\_impl.c221](#)
- 

## System Architecture Mapping

The following diagram maps high-level system concepts to their specific code entities and file locations.

### Natural Language to Code Entity Space

## flowchart LR

```

flowchart LR
 subgraph subGraph1 ["Code Entity Space"]
 A1["AromaNode struct"]
 A2["aroma_node_add_child()"]
 B1["AromaDrawList"]
 B2["AromaDrawTask"]
 C1["AromaWindow"]
 C2["aroma_ui_create_window()"]
 D1["AromaPlatformInterface"]
 D2["aroma_platform_android.c"]
 end
 subgraph subGraph0 ["Natural Language Space"]
 A["'The UI Tree'"]
 B["'Drawing Commands'"]
 C["'The Screen'"]
 D["'Input/OS Layer'"]
 end
 A --> A1
 A --> A2
 B --> B1
 B --> B2
 C --> C1
 C --> C2
 D --> D1
 D --> D2

```

**Sources:**[include/aroma\\_node.h1-20src/core/aroma\\_ui\\_impl.c54-65src/backends/platforms/aroma\\_platform\\_android.c1-50include/aroma\\_drawlist.h1-30](#)

## Technical Terms & Abbreviations

Term	Definition	Code Pointer
<b>ABI</b>	Aroma Backend Interface; the abstraction layer for graphics/platform.	<code>aroma_backend_abi</code> <a href="#">src/backends/aroma_abi.c10-15</a>
<b>Slab Allocator</b>	A memory management system for fixed-size node allocations, critical for embedded targets.	<a href="#">src/core/aroma_slab_alloc.c1-100</a>
<b>Hit-Testing</b>	The process of determining which node an input event (touch/click) belongs to.	<code>aroma_event_hit_test</code> <a href="#">src/core/aroma_event.c50-80</a>
<b>Z-Layer</b>	Integer priority determining the stack order of UI elements.	<code>aroma_node_set_z_index</code> <a href="#">examples/car_infotainment/vehicle/vehicle_view.c180</a>
<b>DP/SP</b>	Density-independent Pixels / Scale-independent Pixels (for fonts).	<code>android_sp_to_px</code> <a href="#">src/backends/platforms/aroma_platform_android.c98-99</a>
<b>Vosk</b>	The offline speech recognition engine used for voice commands.	<code>vosk_model_new</code> <a href="#">examples/car_infotainment/voice/voice_control.c153</a>
<b>Telemetry Bridge</b>	Shared memory interface for IPC (Inter-Process Communication) of vehicle data.	<code>telemetry_bridge_open</code> <a href="#">examples/car_infotainment/core/shared_memory_bridge.h32</a>

## Domain Concepts: Android Integration

AromaUI treats Android as a specific platform backend that utilizes JNI (Java Native Interface) to access system services.

### AromaHelper

The Java-side singleton that handles Android-specific tasks like Bluetooth scanning, Toast messages, and Preferences.

- **Template:** [tools/cli/templates/android/app/src/main/java/AromaHelper.java.tpl1-200](#)
- **JNI Cache:** Native code caches method IDs in `AromaHelperCache` for high-speed calls. [src/backends/platforms/aroma\\_platform\\_android.c133-171](#)

### Native App Glue

The standard Android NDK utility used to manage the `android_app` state and the `ANativeWindow` lifecycle.

- **Pointer:** `g_app` [src/backends/platforms/aroma\\_platform\\_android.c50](#)
  - **Callback:** `aroma_android_set_app` [src/core/aroma\\_ui\\_impl.c31-39](#)
- 

## Data Flow: Event to Action

This diagram illustrates how a physical interaction travels through the system to trigger application logic.

### Event Pipeline Data Flow

## 📄 flowchart LR

```

flowchart LR
 subgraph subGraph2 ["Application Logic"]
 L1["Callback Function"]
 L2["aroma_label_set_text()"]
 end
 end
 subgraph subGraph1 ["Core Event System"]
 E1["AromaEvent Creation"]
 E2["Hit Test (Scene Graph)"]
 E3["Event Bubbling"]
 end
 end
 subgraph subGraph0 ["Platform Backend"]
 P1["OS Event (Touch/Key)"]
 P2["aroma_platform_android.c"]
 end
 end
 P1 --> P2
 P2 -->|"aroma_event_queue()"| E1
 E1 --> E2
 E2 --> E3
 E3 -->|"listener_cb"| L1
 L1 --> L2

```

**Sources:** [src/core/aroma\\_event.c1-100](#)[src/backends/platforms/aroma\\_platform\\_android.c86-114](#)[examples/car\\_infotainment/vehicle/vehicle\\_view.c6-14](#)

## Embedded & Web Terms

- **TFT\_eSPI:** A specific backend for ESP32/embedded displays that uses tiled rendering to fit within small RAM constraints. [src/backends/graphics/aroma\\_graphics\\_tft\\_espi.cpp1-50](#)
- **Emscripten/WASM:** The toolchain and target for running AromaUI in a web browser.
- **DPR:** Device Pixel Ratio handling for high-DPI web screens. [src/core/aroma\\_ui\\_impl.c85-88](#)

- **Fetch:** `emscripten_fetch` used for async tile loading in the Map widget. [src/widgets/aroma\\_map.c21](#)
- **OSRM:** Open Source Routing Machine; the backend API used by `aroma_map.c` to calculate navigation paths. [src/widgets/aroma\\_map.c240-260](#)

**Sources:**

- [src/core/aroma\\_ui\\_impl.c1-250](#)
- [src/backends/platforms/aroma\\_platform\\_android.c1-230](#)
- [include/aroma\\_node.h1-50](#)
- [examples/car\\_infotainment/core/main.c1-125](#)
- [src/widgets/aroma\\_map.c1-260](#)

# API Reference

---

This page provides a high-level index of the public AromaUI API. For detailed documentation on subsystems, see the [Core Framework](#) and [Widget Library](#) sections.

## Core Lifecycle

Function	Description	Source
<code>aroma_ui_init</code>	Initializes the framework, slab allocator, and global state.	<a href="#">include/aroma_ui.h175-183</a>
<code>aroma_ui_shutdown</code>	Releases all resources and resets global state.	<a href="#">include/aroma_ui.h212-217</a>
<code>aroma_ui_is_running</code>	Returns true while the main loop should continue.	<a href="#">include/aroma_ui.h225-230</a>
<code>aroma_ui_process_events</code>	Processes pending input, timer, and system events.	<a href="#">include/aroma_ui.h240-245</a>
<code>aroma_ui_render</code>	Renders a specific window if dirty.	<a href="#">include/aroma_ui.h254-272</a>
<code>aroma_ui_render_all</code>	Renders all active windows.	<a href="#">include/aroma_ui.h277-282</a>

## Window Management

Function	Description	Source
<code>aroma_ui_create_window</code>	Creates a new window with title and dimensions.	<a href="#">include/aroma_ui.h341-356</a>
<code>aroma_ui_destroy_window</code>	Destroys a window and frees resources.	<a href="#">include/aroma_ui.h380-385</a>
<code>aroma_ui_window_set_background</code>	Sets the window background clear color.	<a href="#">include/aroma_ui.h392-397</a>
<code>aroma_ui_window_set_visible</code>	Shows or hides a window.	<a href="#">include/aroma_ui.h404-409</a>
<code>aroma_ui_window_count</code>	Returns the number of managed windows.	<a href="#">include/aroma_ui.h415-418</a>
<code>aroma_ui_get_window_at</code>	Retrieves a window by index.	<a href="#">include/aroma_ui.h425-430</a>

## Theming and Fonts

Function	Description	Source
<code>aroma_ui_set_theme</code>	Sets the global theme for all widgets.	<a href="#">include/aroma_ui.h189-196</a>
<code>aroma_ui_get_theme</code>	Returns the current global theme.	<a href="#">include/aroma_ui.h202-205</a>
<code>aroma_ui_load_font</code>	Loads a font from a filesystem path.	<a href="#">include/aroma_ui.h290-308</a>
<code>aroma_ui_unload_font</code>	Destroys a loaded font.	<a href="#">include/aroma_ui.h314-321</a>
<code>aroma_ui_prepare_font_for_window</code>	Binds a font to a window for rendering.	<a href="#">include/aroma_ui.h323-328</a>

# Widget Factory Functions

## Layout and Navigation

Helper	Underlying Factory	Description	Source
<code>aroma_ui_container</code>	<code>aroma_container_create</code>	Creates a container with flex/ grid layout.	<a href="#">include/ aroma_ui.h602-</a>
<code>aroma_ui_scrollable_container</code>	<code>aroma_container_create</code>	Creates a scrollable flex container.	<a href="#">include/ aroma_ui.h636-</a>
<code>aroma_ui_listview</code>	<code>aroma_listview_create</code>	Creates a list view inside a scroll container.	<a href="#">include/ aroma_ui.h1043-</a>
<code>aroma_ui_table</code>	<code>aroma_table_create</code>	Creates a scrollable data table.	<a href="#">include/ aroma_ui.h108-</a>
<code>aroma_ui_tabs</code>	<code>aroma_tabs_create</code>	Creates a horizontal tab bar.	<a href="#">include/ aroma_ui.h117-</a>
<code>aroma_ui_tabs_with_icons</code>	<code>aroma_tabs_set_icon</code>	Creates tabs with icon support.	<a href="#">include/ aroma_ui.h119-</a>
<code>aroma_ui_sidebar</code>	<code>aroma_sidebar_create</code>	Creates a vertical sidebar navigation.	<a href="#">include/ aroma_ui.h128-</a>
<code>aroma_ui_sidebar_with_icons</code>	<code>aroma_sidebar_set_icon</code>	Creates a sidebar	<a href="#">include/ aroma_ui.h130-</a>

Helper	Underlying Factory	Description	Source
		with icon support.	

## Input and Control

Helper	Underlying Factory	Description	Source
<code>aroma_ui_button</code>	<code>aroma_button_create</code>	Creates a text button.	<a href="#">include/aroma_ui.h514-53</a>
<code>aroma_ui_button_with_icon</code>	<code>aroma_button_set_icon</code>	Creates a button with a leading icon.	<a href="#">include/aroma_ui.h556-57</a>
<code>aroma_ui_iconbutton</code>	<code>aroma_iconbutton_create</code>	Creates an icon-only button (FAB, toolbar).	<a href="#">include/aroma_ui.h888-90</a>
<code>aroma_ui_checkbox</code>	<code>aroma_checkbox_create</code>	Creates a checkbox with label.	<a href="#">include/aroma_ui.h720-73</a>
<code>aroma_ui_radiobutton</code>	<code>aroma_radiobutton_create</code>	Creates a radio button in a group.	<a href="#">include/aroma_ui.h755-77</a>
<code>aroma_ui_switch</code>	<code>aroma_switch_create</code>	Creates a toggle switch.	<a href="#">include/aroma_ui.h789-80</a>
<code>aroma_ui_slider</code>	<code>aroma_slider_create</code>	Creates a range slider.	<a href="#">include/aroma_ui.h821-83</a>
<code>aroma_ui_textbox</code>	<code>aroma_textbox_create</code>	Creates a single-line text input.	<a href="#">include/aroma_ui.h852-87</a>

Helper	Underlying Factory	Description	Source
<code>aroma_ui_dropdown</code>	<code>aroma_dropdown_create</code>	Creates a dropdown select input.	<a href="#">include/aroma_ui.h1341-1</a>
<code>aroma_ui_chip</code>	<code>aroma_chip_create</code>	Creates a filter/choice chip.	<a href="#">include/aroma_ui.h922-94</a>
<code>aroma_ui_chip_with_icon</code>	<code>aroma_chip_set_icon</code>	Creates a chip with a leading icon.	<a href="#">include/aroma_ui.h942-95</a>

## Display and Feedback

Helper	Underlying Factory	Description	Source
<code>aroma_ui_label</code>	<code>aroma_label_create</code>	Creates a text label.	<a href="#">include/aroma_ui.h58</a>
<code>aroma_ui_icon</code>	<code>aroma_icon_create</code>	Creates a standalone icon glyph.	<a href="#">include/aroma_ui.h68</a>
<code>aroma_ui_image</code>	<code>aroma_image_create</code>	Creates an image from a file path.	<a href="#">include/aroma_ui.h65</a>
<code>aroma_ui_image_mem</code>	<code>aroma_image_create_from_memory</code>	Creates an image from a memory buffer.	<a href="#">include/aroma_ui.h66</a>
<code>aroma_ui_card</code>	<code>aroma_card_create</code>	Creates a styled card container.	<a href="#">include/aroma_ui.h97</a>
<code>aroma_ui_progressbar</code>	<code>aroma_progressbar_create</code>	Creates a linear progress indicator.	<a href="#">include/aroma_ui.h99</a>
<code>aroma_ui_loading</code>	<code>aroma_loading_create</code>	Creates a spinning loader.	<a href="#">include/aroma_ui.h13</a>
<code>aroma_ui_divider</code>	<code>aroma_divider_create</code>	Creates a horizontal or vertical divider.	<a href="#">include/aroma_ui.h10</a>
<code>aroma_ui_dialog</code>	<code>aroma_dialog_create</code>		<a href="#">include/aroma_ui.h11</a>

Helper	Underlying Factory	Description	Source
		Creates a modal dialog.	
<code>aroma_ui_snackbar</code>	<code>aroma_snackbar_create</code>	Creates a transient bottom notification.	<a href="#">include/aroma_ui.h12</a>
<code>aroma_ui_tooltip</code>	<code>aroma_tooltip_create</code>	Creates a floating tooltip.	<a href="#">include/aroma_ui.h12</a>
<code>aroma_ui_menu</code>	<code>aroma_menu_create</code>	Creates a floating context menu.	<a href="#">include/aroma_ui.h11</a>
<code>aroma_ui_debug_overlay</code>	<code>aroma_debug_overlay_create</code>	Creates a runtime debug overlay.	<a href="#">include/aroma_ui.h13</a>

## Specialized Widgets

Helper	Underlying Factory	Description	Source
<code>aroma_ui_map</code>	<code>aroma_map_create</code>	Creates an interactive OpenStreetMap widget.	<a href="#">include/aroma_ui.h153-158</a>
<code>aroma_ui_3d_viewer</code>	<code>aroma_3d_viewer_create</code>	Creates a 3D model viewer widget.	<a href="#">include/widgets/aroma_3d_viewer.h14</a>

## Node Manipulation

Function	Description	Source
<code>aroma_node_set_hidden</code>	Shows or hides a node and its subtree.	<a href="#">src/core/aroma_node.c1012</a>
<code>aroma_node_set_z_index</code>	Sets the draw order priority.	<a href="#">include/aroma_node.h95-98</a>
<code>aroma_node_set_layout_mode</code>	Sets container layout mode (NONE, FLEX, GRID).	<a href="#">src/core/aroma_layout.c59-61</a>
<code>aroma_node_set_layout_fill</code>	Makes a node fill its parent bounds.	<a href="#">docs/ui/layouts.md41-54</a>
<code>aroma_node_set_layout_center</code>	Centers a node within its parent.	<a href="#">docs/ui/layouts.md58-71</a>
<code>aroma_node_set_layout_anchor</code>	Pins a node to parent edges.	<a href="#">docs/ui/layouts.md75-100</a>
<code>aroma_node_invalidate</code>	Marks a node as dirty for the next render.	<a href="#">src/core/aroma_ui_impl.c221</a>
<code>aroma_node_set_draw_cb</code>	Assigns a custom draw callback to a node.	<a href="#">include/aroma_node.h160-165</a>

## Animation

Function	Description	Source
<code>aroma_animation_start</code>	Starts a standard property animation.	<a href="#">src/core/aroma_animation.c121-142</a>
<code>aroma_animation_start_custom</code>	Starts a custom animation with a user callback.	<a href="#">src/core/aroma_animation.c155-162</a>
<code>aroma_animation_stop</code>	Stops all animations on a target node.	<a href="#">src/core/aroma_animation.c144-152</a>

## Animation Types

Type	Property Modified
<code>AROMA_ANIM_SLIDE_X</code>	<code>node-&gt;rect.x</code>
<code>AROMA_ANIM_SLIDE_Y</code>	<code>node-&gt;rect.y</code>
<code>AROMA_ANIM_SCALE_X</code>	<code>node-&gt;rect.width</code>
<code>AROMA_ANIM_SCALE_Y</code>	<code>node-&gt;rect.height</code>
<code>AROMA_ANIM_FADE</code>	<code>node-&gt;opacity</code>
<code>AROMA_ANIM_CUSTOM</code>	User-defined callback

## Easing Functions

Easing	Description
<code>AROMA_EASE_LINEAR</code>	Constant speed.
<code>AROMA_EASE_IN_QUAD</code>	Quadratic acceleration.
<code>AROMA_EASE_OUT_QUAD</code>	Quadratic deceleration.
<code>AROMA_EASE_IN_OUT_QUAD</code>	Quadratic acceleration then deceleration.
<code>AROMA_EASE_OUT_CUBIC</code>	Smooth deceleration (default).
<code>AROMA_EASE_OUT_BACK</code>	Slight overshoot before settling.
<code>AROMA_EASE_OUT_ELASTIC</code>	Damped oscillation effect.

**Sources:**[include/aroma\\_animation.h10-28src/core/aroma\\_animation.c45-78](#)

## Event System

Function	Description	Source
<code>aroma_event_subscribe</code>	Registers a listener for a node and event type.	<a href="#">src/core/aroma_event.c231-260</a>
<code>aroma_event_unsubscribe</code>	Removes a listener.	<a href="#">src/core/aroma_event.c261-280</a>
<code>aroma_event_create_mouse</code>	Creates a synthetic mouse event.	<a href="#">src/core/aroma_event.c290-320</a>
<code>aroma_event_queue</code>	Queues an event for processing.	<a href="#">src/core/aroma_event.c510-530</a>
<code>aroma_event_process_queue</code>	Drains the event queue.	<a href="#">src/core/aroma_event.c510-530</a>

## Event Types

Event	Description
<code>EVENT_TYPE_MOUSE_MOVE</code>	Pointer moved.
<code>EVENT_TYPE_MOUSE_CLICK</code>	Primary button pressed.
<code>EVENT_TYPE_MOUSE_RELEASE</code>	Primary button released.
<code>EVENT_TYPE_MOUSE_SCROLL</code>	Scroll wheel moved.
<code>EVENT_TYPE_MOUSE_ENTER</code>	Pointer entered node bounds.
<code>EVENT_TYPE_MOUSE_EXIT</code>	Pointer left node bounds.
<code>EVENT_TYPE_KEY_PRESS</code>	Keyboard key pressed.
<code>EVENT_TYPE_KEY_RELEASE</code>	Keyboard key released.
<code>EVENT_TYPE_TOUCH_DOWN</code>	Touch contact began.
<code>EVENT_TYPE_TOUCH_MOVE</code>	Touch contact moved.
<code>EVENT_TYPE_TOUCH_UP</code>	Touch contact ended.
<code>EVENT_TYPE_TOUCH_CANCEL</code>	Touch contact was cancelled.

**Sources:**[include/aroma\\_event.h40-80src/core/aroma\\_event.c146-160](#)

## Android Platform

Function	Description	Source
<code>aroma_android_set_app</code>	Sets the Android app state for JNI.	<a href="#">src/core/aroma_ui_impl.c31-39</a>
<code>aroma_ui_android_intent</code>	Sends an Android Intent (VIEW, SEND, DIAL, CALL).	<a href="#">include/aroma_ui.h490-496</a>

**Sources:**[include/aroma\\_android.h89-204](#)

---